## Table of Contents

---

# 1. Introduction

This document describes a GitHub App that reviews pull requests using AWS Bedrock and Lambda. The authored functional architecture remains stable while deployment/runtime/code `evidence` is inferred from infrastructure manifests, source files, and verification artifacts.

## 1.1. Scope and Assumptions

- `Functional Groups` and `Functional Units` are the stable authored design backbone.
- `Runtime elements` and `code elements` are inferred from available IaC, deployment, and source artifacts.
- Inferred coverage is evidence-driven and may be partial when source metadata is missing.
- `Requirements` and reference indexes are provided for traceability, not as primary narrative flow.

# 2. How To Read This Document

This document is intentionally split into two layers:

1. Concept layer: functional and view-specific architecture narratives for human understanding.
2. Evidence layer: generated requirement traceability and inferred reference indexes.

Read in this order:

1. Architecture Intent View: stable authored architecture intent and ownership boundaries.
2. Communication / Deployment / Security Views: viewpoint-specific operational behavior.
3. Traceability View: requirement mapping, evidence coverage, and confidence.
4. Reference Appendix: supporting reference IDs and inferred-source links.

## 3. Document Health Snapshot

| View | Authored Status | Authored Status Explanation | Units in Scope | Direct Evidence Coverage | Gap Count | Heuristic Confidence | Heuristic Basis |
|---|---|---|---|---|---|---|---|
| Architecture Intent View (`VIEW-ARCHITECTURE-INTENT`) | stable | Authored capability boundaries and ownership responsibilities are stable for this release. | 7 | 7 | 0 | high | Defaults authored units to covered; use authoredStatus as the primary publication signal. |
| Communication View (`VIEW-COMMUNICATION`) | in-review | Communication sequencing is defined, while protocol-level metadata is still being expanded. | 7 | 3 | 4 | low | Counts a unit as covered when direct inferred runtime evidence exists. |
| Deployment View (`VIEW-DEPLOYMENT`) | in-review | Deployment and runtime mapping are present; observability and ownership evidence are still evolving. | 7 | 3 | 4 | low | Counts a unit as covered when direct inferred runtime evidence exists. |
| Security View (`VIEW-SECURITY`) | draft | Core threats are modeled; control-depth and detection mappings are still being expanded. | 7 | 7 | 2 | high | Counts a unit as covered when explicit authored attack-vector mapping exists. |
| Traceability View (`VIEW-TRACEABILITY`) | in-review | Requirement mappings exist; evidence completeness and consistency are still being hardened. | 7 | 7 | 4 | high | Counts a unit as covered when direct inferred runtime or code evidence exists. |
| State Lifecycle View (`VIEW-STATE-LIFECYCLE`) | in-review | Review lifecycle states and guarded transitions are explicitly modeled. | 0 | 0 | 0 | n/a | Lifecycle heuristic is currently basic; treat authoredStatus as primary signal. |
| Interaction Flow View (`VIEW-INTERACTION-FLOW`) | in-review | Interaction flow captures PR review causality from ingress through publication. | 4 | 4 | 0 | high | Flow heuristic is step/branch-oriented; treat authoredStatus plus projection completeness as the primary signal. |

# 4. Terms and Definitions

| Term | Definition |
| --- | --- |
| actor (TERM-ACTOR) | A person or operational role that interacts with functional units. |
| ai review is enabled (FEAT-AI-REVIEW) | Feature flag enabling model-assisted review. Aliases:ai review enabled |
| architecture decision (TERM-DECISION) | Authored architecture decision record with status, context, decision text, and consequences. |
| AsciiDoc diagram (TERM-ASCIIDOC-DIAGRAM) | Generated diagram block, usually Mermaid, that visualizes architecture relationships in the AsciiDoc publication. |
| AsciiDoc document (TERM-ASCIIDOC-DOCUMENT) | Human-readable generated architecture publication document assembled from authored model, inferred evidence, diagrams, references, and decisions. |
| AsciiDoc section (TERM-ASCIIDOC-SECTION) | Structured generated document section or row model used to render authored and inferred architecture content. |
| attack vector (TERM-ATTACK-VECTOR) | A technical misuse or attack path that targets functional, referenced, or runtime elements. |
| audit logging is enabled (FEAT-AUDIT-LOGGING) | Feature flag enabling audit metadata persistence. Aliases:audit enabled |
| authored mapping (TERM-AUTHORED-MAPPING) | An explicit relationship declared in architecture inputs between authored or referenced elements. |
| aws sdk for go (REF-AWS-SDK-GO) | SDK dependency used for AWS runtime integrations. Aliases:aws sdk |
| aws secrets manager (REF-AWS-SECRETS-MANAGER) | Managed secret store for application credentials and config values. Aliases:secrets manager |
| bedrock runtime (REF-AWS-BEDROCK-RUNTIME) | External Bedrock runtime endpoint used for model inference. Aliases:bedrock api |
| bedrock runtime is unavailable (EVT-BEDROCK-UNAVAILABLE) | Event raised when Bedrock inference path is degraded. Aliases:bedrock unavailable |
| catalog (TERM-CATALOG) | Controlled vocabulary document used to normalize model, requirement, and generated-document terminology. |
| catalog entry (TERM-CATALOG-ENTRY) | A stable catalog term with a definition and aliases for linting, linking, and generated references. |
| check run summary (DATA-CHECKRUN-SUMMARY) | Aggregated review result posted as a GitHub check run. Aliases:check run summaries |
| CLI command (TERM-CLI-COMMAND) | Command-line entrypoint that coordinates loading, validation, generation, and file output for an engineering-model workflow. |

| Term | Definition |
| --- | --- |
| `code element` (TERM-CODE-ELEMENT) | An inferred code structure or ownership element discovered from source trees and build metadata. |
| `compliance mapping` (TERM-COMPLIANCE-MAPPING) | Authored mapping that links a model control to selected compliance controls, model scope, implementation status, evidence, and responsible roles. |
| `control` (TERM-CONTROL) | An authored security, policy, or operational control used to constrain behavior and reduce risk. |
| `control verification` (TERM-CONTROL-VERIFICATION) | Authored control verification record with method, status, threat/risk scope, findings, and evidence. |
| `data object` (TERM-DATA-OBJECT) | An authored data artifact or contract shape traced across flow, interface, and implementation boundaries. |
| `deployment target` (TERM-DEPLOYMENT-TARGET) | An authored deployment destination such as environment, cluster, namespace, or registry scope. |
| `design document` (TERM-DESIGN) | Authored design narrative organized by model entities and architecture views. |
| `design view` (TERM-DESIGN-VIEW) | View-scoped design narrative attached to an authored functional group or functional unit. |
| `developer` (ACT-DEVELOPER) | Engineer opening and updating pull requests. Aliases:pr author |
| `diff context` (DATA-DIFF-CONTEXT) | Pull request file changes and metadata prepared for analysis. Aliases:normalized review context, pull request files, pull request metadata, review context, review-ready context |
| `diff context is prepared` (EVT-CONTEXT-PREPARED) | Event indicating context assembly completed and is ready for analysis. Aliases:context prepared |
| `event` (TERM-EVENT) | An authored trigger signal that drives transitions, flow progress, or asynchronous outcomes. |
| `evidence` (TERM-EVIDENCE) | A file, artifact, or observation used to support control, risk, threat, or verification claims. |
| `flow` (TERM-FLOW) | An authored causal interaction sequence from user/system intent to outcome, represented as ordered steps. |
| `flow step` (TERM-FLOW-STEP) | A single authored step in a flow that captures action, data in/out, references, and normal/error/async transitions. |
| `functional group` (TERM-FUNCTIONAL-GROUP) | A major authored capability area that groups related functional units. |
| `functional unit` (TERM-FUNCTIONAL-UNIT) | An authored working unit inside a functional group that owns specific behavior. |
| `generation workflow` (TERM-GENERATION-WORKFLOW) | Orchestrated run that combines model loading, validation, projection, rendering, and artifact emission. |
| `github app api` (REF-GITHUB-APP-API) | External GitHub API used for pull request metadata and review publication. Aliases:github api |

| Term | Definition |
| --- | --- |
| `github rest sdk` (REF-GITHUB-REST-SDK) | SDK dependency used for GitHub App REST interactions. Aliases:github sdk |
| `github webhook ingress` (FU-GITHUB-WEBHOOK-INGRESS) | Validates webhook signatures and routes trusted pull request events. Aliases:webhook ingress |
| `inference hint` (TERM-INFERENCE-HINT) | Authored source and ownership configuration used to discover runtime, code, and verification evidence. |
| `inferred mapping` (TERM-INFERRED-MAPPING) | A discovered relationship that links inferred runtime/code elements upward to authored design. |
| `interface` (TERM-INTERFACE) | An authored interface boundary (for example API, channel, endpoint, or contract) owned by a functional unit. |
| `lambda runtime operations` (FU-LAMBDA-RUNTIME-OPERATIONS) | Manages runtime deployment and operational controls. Aliases:lambda ops |
| `lambda runtime operations are enabled` (FEAT-LAMBDA-RUNTIME-OPS) | Feature flag enabling Lambda runtime deployment lifecycle operations. Aliases:lambda runtime ops enabled |
| `lint run` (TERM-LINT-RUN) | Requirement lint configuration that controls parsing and quality checks for requirement text. |
| `LOBSTER activity trace` (TERM-LOBSTER-ACTIVITY-TRACE) | Generated LOBSTER activity JSON that links verification evidence back to formal requirement identifiers. |
| `MCP server` (TERM-MCP-SERVER) | Model Context Protocol server that exposes engineering-model operations to AI agents through JSON-RPC tools. |
| `MCP stdio transport` (TERM-MCP-STDIO-TRANSPORT) | Content-Length framed standard input/output transport used by the MCP command-line server. |
| `MCP tool` (TERM-MCP-TOOL) | Named MCP operation with a constrained input schema and structured response payload. |
| `MCP tool response` (TERM-MCP-TOOL-RESPONSE) | Structured MCP tool payload that includes success state, schema version, generated timestamp, and result data or validation error. |
| `model` (TERM-MODEL) | The authored architecture model root that composes functional structure, relationships, inference hints, and views. |
| `model bundle` (TERM-BUNDLE) | Loaded architecture, catalog, and companion documents resolved as one working model context. |
| `normal mode is active` (MODE-NORMAL) | Default operating mode using AI plus deterministic checks. Aliases:normal mode |
| `Open OTM document` (TERM-OPEN-OTM-DOCUMENT) | Open Threat Model JSON representation generated from the engineering model for interoperable threat-model exchange. |
| `OSCAL assessment results` (TERM-OSCAL-ASSESSMENT-RESULTS) | Generated OSCAL assessment results that summarize reviewed controls, verification findings, and modeled risks. |

| Term | Definition |
| --- | --- |
| `OSCAL POA&M` (TERM-OSCAL-POAM) | Generated OSCAL plan of action and milestones that maps modeled POA&M items and related risks into compliance JSON. |
| `OSCAL SSP` (TERM-OSCAL-SSP) | Generated OSCAL system security plan that maps authored system metadata, components, controls, allocations, and evidence into SSP JSON. |
| `platform operations` (FG-PLATFORM-OPERATIONS) | Functional group for deployment lifecycle and secret/config controls. Aliases:runtime operations |
| `platform operator` (ACT-PLATFORM-OPERATOR) | Owner of deployment/runtime operations and reliability controls. Aliases:platform ops |
| `POA&M item` (TERM-POAM-ITEM) | Plan of action and milestones item linked to a modeled risk and supporting artifacts. |
| `policy checks` (FU-POLICY-CHECKS) | Evaluates deterministic repository policy checks. Aliases:deterministic checks |
| `policy checks are enabled` (FEAT-POLICY-CHECKS) | Feature flag enabling deterministic policy checks. Aliases:policy checks enabled |
| `policy ruleset` (REF-POLICY-RULESET) | Deterministic review policy bundle used during analysis. Aliases:policy bundle |
| `pr context assembly` (FU-PR-CONTEXT-ASSEMBLY) | Fetches changed files and prepares bounded review context payloads. Aliases:context assembly |
| `pr integration` (FG-PR-INTEGRATION) | Functional group for webhook ingress and pull request context intake. Aliases:github integration |
| `pr review system` (SYS-PR-REVIEW-SYSTEM) | GitHub App review system-of-interest that ingests PR events and publishes review outcomes. Aliases:the pr review system |
| `prompt injection in diff` (AV-PROMPT-INJECTION-IN-DIFF) | Malicious prompt-like content embedded in pull request code changes. Aliases:prompt injection |
| `pull request opened event is received` (EVT-PR-OPENED) | Event raised when a new pull request is opened. Aliases:pr opened, webhook events |
| `pull request review is active` (STATE-PR-REVIEW-ACTIVE) | State where review processing is currently active for a pull request. Aliases:review active |
| `pull request spam abuse` (AV-PR-SPAM-ABUSE) | High-volume event abuse causing review capacity exhaustion. Aliases:pr spam |
| `pull request synchronize event is received` (EVT-PR-SYNCHRONIZE) | Event raised when new commits are pushed to an open pull request. Aliases:authorized events, pr synchronize |
| `redacted audit metadata` (DATA-REDACTED-AUDIT-METADATA) | Sanitized prompt and finding metadata retained for audit. |
| `reference index` (TERM-REFERENCE-INDEX) | Generated index of authored, catalog, runtime, code, and verification references with stable anchors and backlinks. |

| Term | Definition |
|---|---|
| `referenced element` (TERM-REFERENCED-ELEMENT) | An architecture-relevant external, platform, or third-party dependency represented by role. |
| `release artifact is approved` (EVT-RELEASE-APPROVED) | Event raised when a Lambda runtime release artifact has passed approval. Aliases:release approved |
| `repository index` (TERM-REPO-INDEX) | Bounded first-party source index used to answer model-aware file, requirement, control, and threat lookup queries. |
| `repository maintainer` (ACT-REPOSITORY-MAINTAINER) | Owner of merge decisions and repository quality gates. Aliases:maintainer |
| `requirement` (TERM-REQUIREMENT) | A structured requirement statement that defines expected system behavior and maps to functional ownership. |
| `result metadata` (DATA-RESULT-METADATA) | Metadata describing review output characteristics for audit and reporting. |
| `review findings are generated` (EVT-FINDINGS-GENERATED) | Event indicating analysis output is ready for publication. Aliases:findings generated, review outcome |
| `review findings are published` (EVT-FINDINGS-PUBLISHED) | Event indicating review output has been published to GitHub checks/comments. Aliases:findings published, publication output |
| `review intelligence` (FG-REVIEW-INTELLIGENCE) | Functional group for AI and deterministic review analysis and publication. Aliases:review system |
| `review orchestration` (FU-REVIEW-ORCHESTRATION) | Coordinates Bedrock analysis with deterministic checks and fallback logic. Aliases:orchestration |
| `review publication` (FU-REVIEW-PUBLICATION) | Publishes review findings to GitHub checks and comments. Aliases:publication |
| `review request rate limit is exceeded` (EVT-RATE-LIMIT-EXCEEDED) | Event raised when PR review rate guardrails are hit. Aliases:rate limit exceeded |
| `risk` (TERM-RISK) | Authored risk record with likelihood, impact, response, scope, controls, and evidence. |
| `runtime element` (TERM-RUNTIME-ELEMENT) | An inferred runtime realization element discovered from infrastructure and deployment sources. |
| `secret leakage in review output` (AV-SECRET-LEAKAGE-IN-REVIEW) | Unintended credential disclosure in generated review findings. Aliases:secret leakage |
| `secrets and configuration` (FU-SECRETS-CONFIGURATION) | Manages credentials and secure runtime configuration. Aliases:secret config |
| `security context` (TERM-SECURITY-CONTEXT) | Security-focused generated context view that groups owned functional units and shows external actors, references, flows, controls, and trust boundaries. |
| `security engineer` (ACT-SECURITY-ENGINEER) | Owner of security policy checks and finding triage. Aliases:security reviewer |

| Term | Definition |
| --- | --- |
| `source block` (TERM-SOURCE-BLOCK) | Stable source reference block that records file path, optional line span, kind, summary, and linked entity IDs. |
| `spoofed github webhook` (AV-SPOOFED-GITHUB-WEBHOOK) | Forged webhook request attempting unauthorized review execution. Aliases:webhook spoofing |
| `state` (TERM-STATE) | An authored lifecycle state used to model transition behavior, guards, and event-driven progression. |
| `Structurizr DSL` (TERM-STRUCTURIZR-DSL) | Generated Structurizr DSL workspace that projects authored architecture, relationships, dynamic views, and deployment metadata. |
| `threat assumption` (TERM-THREAT-ASSUMPTION) | Authored security assumption that records scope, rationale, owner, status, and evidence. |
| `Threat Dragon document` (TERM-THREAT-DRAGON-DOCUMENT) | Threat Dragon JSON representation generated from the engineering model for STRIDE threat-model review. |
| `threat mitigation` (TERM-THREAT-MITIGATION) | Authored mitigation record that links a threat scenario to a control, effectiveness, verification, and evidence. |
| `threat model export` (TERM-THREAT-MODEL-EXPORT) | Generated security artifact that translates authored architecture, flows, trust boundaries, threats, and mitigations into an external threat-model schema. |
| `threat out-of-scope decision` (TERM-THREAT-OUT-OF-SCOPE) | Authored decision that excludes a threat concern from current scope with reason, owner, expiry, and evidence. |
| `threat scenario` (TERM-THREAT-SCENARIO) | Authored threat narrative that connects attack vectors, scope, flows, controls, risks, and verification evidence. |
| `trace marker` (TERM-TRACE-MARKER) | Source-level marker such as TRLC-LINKS or ENGMODEL-LINKS used to connect declarations to requirements and model entities. |
| `TRLC package` (TERM-TRLC-PACKAGE) | Generated TRLC model and requirements package used for formal requirement traceability processing. |
| `trust boundary` (TERM-TRUST-BOUNDARY) | An authored trust separation zone that marks policy, identity, or security control boundaries. |
| `upward linking` (TERM-UPWARD-LINKING) | Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs. |
| `validation` (TERM-VALIDATION) | Model quality gate that checks authored documents, references, relationship semantics, and requirement lint results. |
| `validation diagnostic` (TERM-VALIDATION-DIAGNOSTIC) | Structured validation finding with code, severity, message, and optional source path. |
| `verification check` (TERM-VERIFICATION-CHECK) | An inferred or authored verification artifact that validates requirement behavior with test evidence. |

| Term | Definition |
| --- | --- |
| `view` (TERM-VIEW) | Authored projection configuration that selects roots, entity kinds, mappings, audience, and abstraction. |
| `webhook signature` (DATA-WEBHOOK-SIGNATURE) | Signature used to verify webhook authenticity. |
| `webhook signature is valid` (COND-WEBHOOK-SIGNATURE-VALID) | Condition indicating webhook HMAC verification succeeded. Aliases:signature valid |

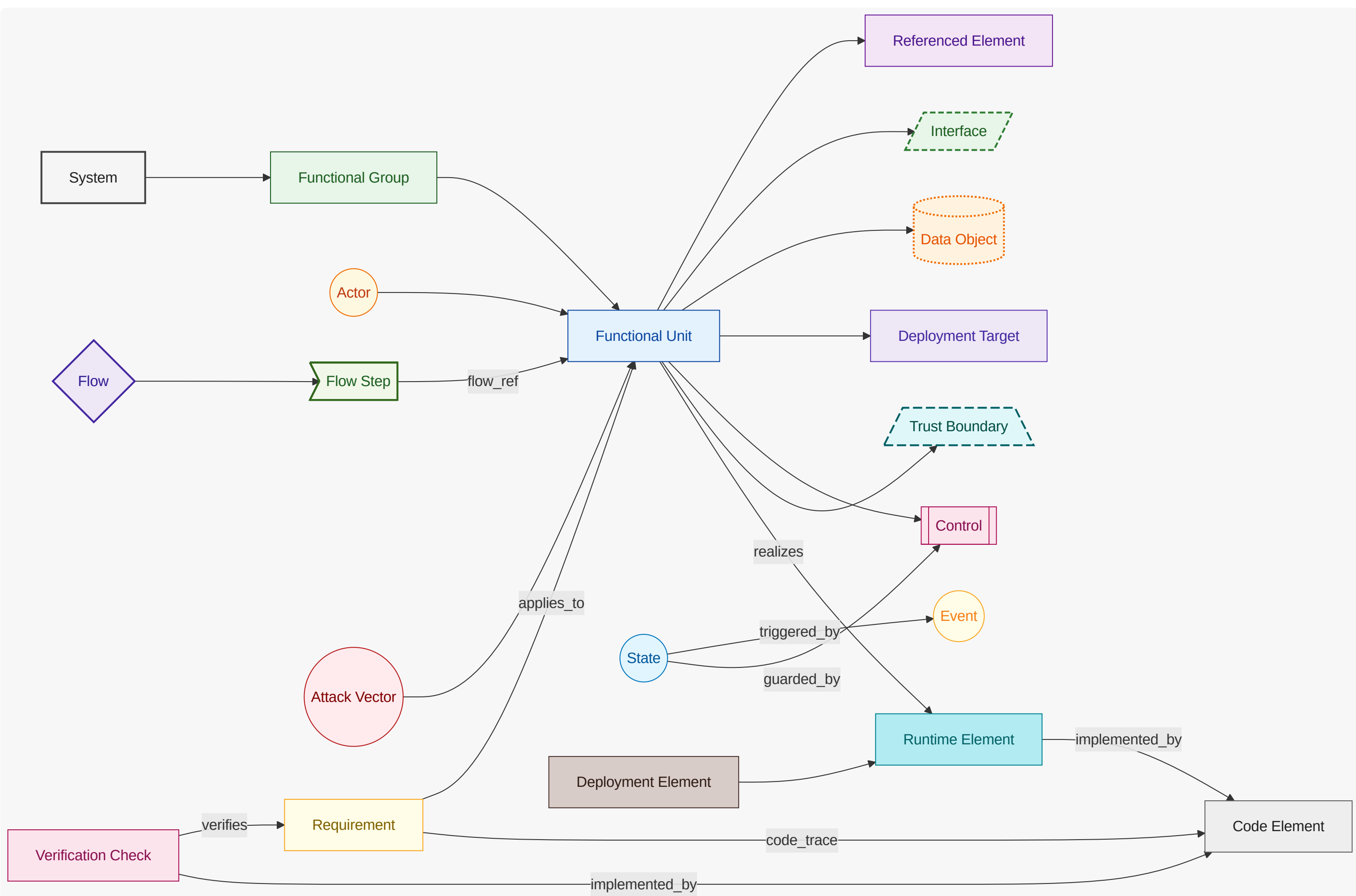
## 5. Architecture Decision Records

Architecture decision records are generated separately and linked here as a compact index.

| ID | Title | Status | Date |
| --- | --- | --- | --- |
| ADR-PRR-001 | Separate webhook ingress from review orchestration | accepted | 2026-04-29 |

### 5.1. Diagram Color and Shape Legend

This legend applies to all Mermaid diagrams in this document.



# 6. Architecture Intent View

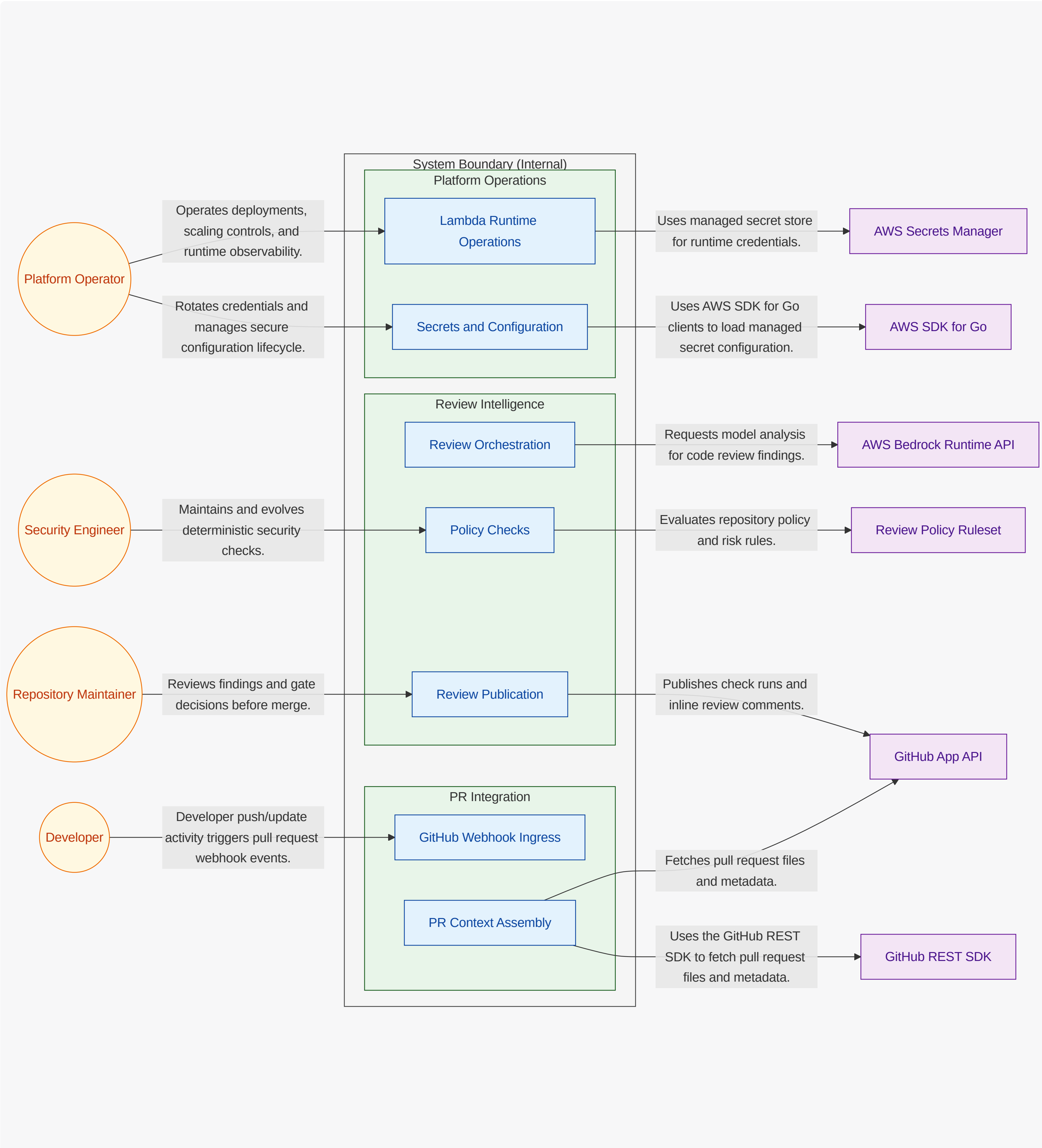
Show the stable authored architecture intent: capability areas, unit ownership boundaries, and intentional responsibility split.

## 6.1. What This View Answers

- What are the main capability areas?
- What are the stable units of responsibility and what does each unit own?
- How are responsibilities intentionally split?

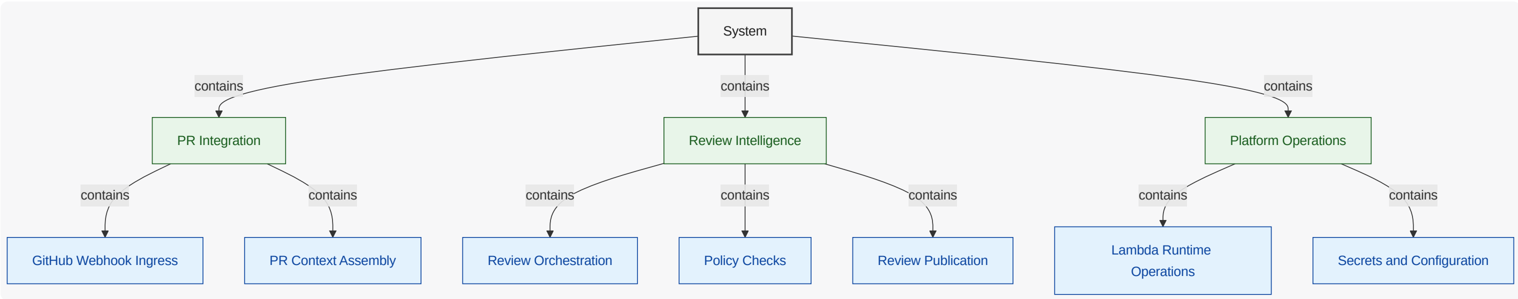
## 6.2. System Boundary Diagram

What this diagram shows: explicit internal vs external scope. Functional units are inside the `System Boundary (Internal)` container, while actors and referenced dependencies are shown outside the boundary.




## 6.3. System Decomposition Diagram

What this diagram shows: stable capability decomposition from functional groups into functional units.


System
contains
contains
contains
PR Integration
Review Intelligence
Platform Operations
contains
contains
contains
contains
contains
contains
contains
GitHub Webhook Ingress
PR Context Assembly
Review Orchestration
Policy Checks
Review Publication
Lambda Runtime Operations
Secrets and Configuration

## 6.4. Functional Group Manhattan Matrix

What this diagram shows: a table-like matrix where functional groups are the bottom row (columns) and functional units are arranged in rows above their owning group.

| | | |
|---|---|---|
| | Policy Checks | |
| GitHub Webhook Ingress | Review Orchestration | Lambda Runtime Operations |
| PR Context Assembly | Review Publication | Secrets and Configuration |
| PR Integration | Review Intelligence | Platform Operations |

## 6.5. Functional Groups and Units (View Scoped)

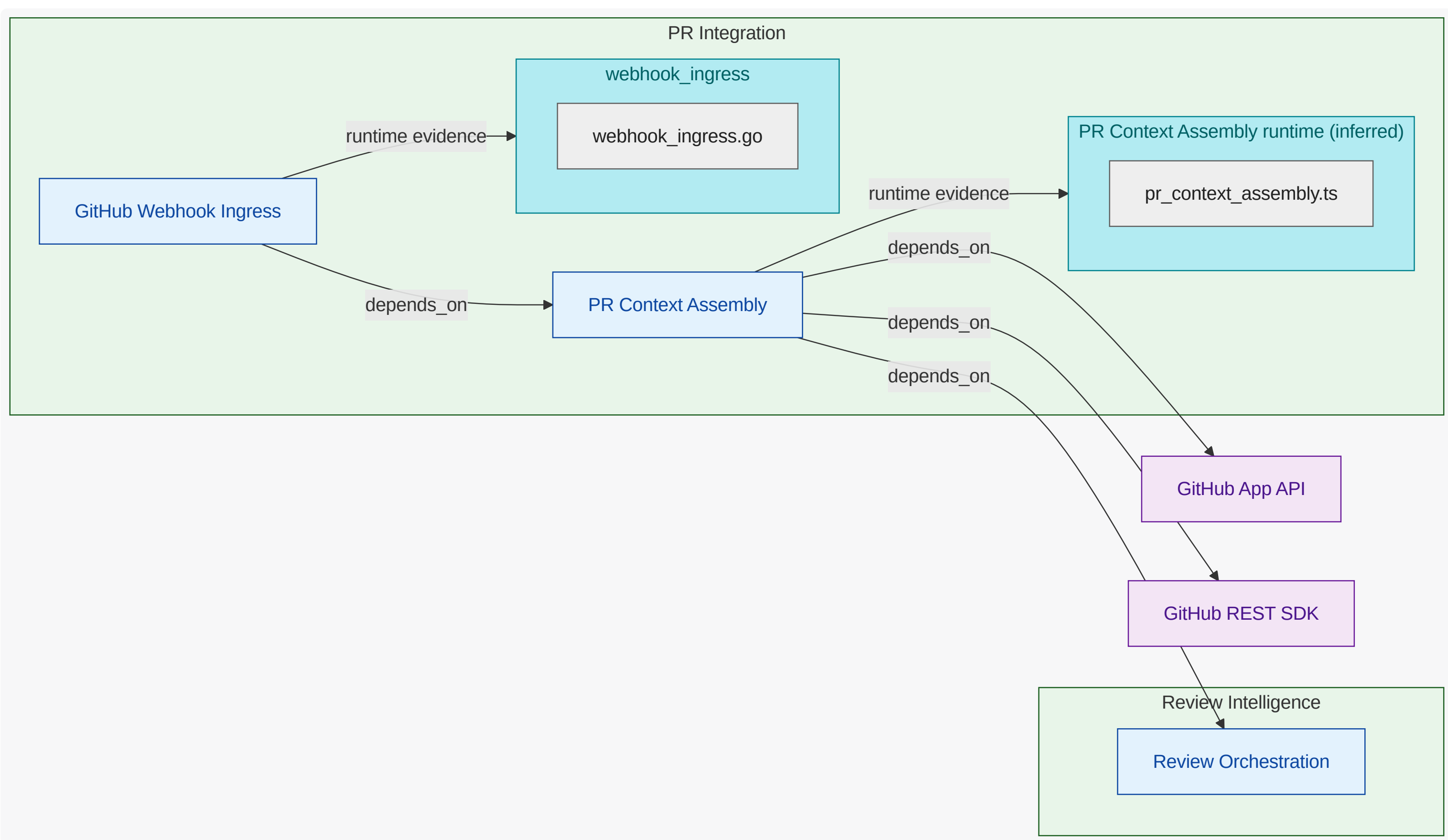
### 6.5.1. PR Integration (FG-PR-INTEGRATION)

Ingress and `context assembly` for GitHub pull request `events`.

```
    <<REF_IDX-CATALOG_FG_PR_INTEGRATION,PR Integration>> owns webhook <<REF_IDX-ENGMODEL_TERM_TRUST_BOUNDARY,trust boundaries>>
and <<REF_IDX-DO_DO_BEDROCK_PR_CONTEXT,pull request context>> intake.
It converts incoming GitHub <<REF_IDX-ENGMODEL_TERM_EVENT,events>> into normalized review work with clear boundaries.
```

#### Functional Unit Dependency Diagram

What this diagram shows: dependencies involving functional units in this functional group. Units in this group are rendered inside the group container; connected external units and referenced elements are shown outside it.




#### GitHub Webhook Ingress (FU-GITHUB-WEBHOOK-INGRESS)

Verifies `webhook signatures`, validates `event` type, and starts review workflow routing.

##### Owned Responsibility

functional responsibility for `github webhook ingress`; includes decision and `orchestration` `flow` to `PR Context Assembly`

#### Key Collaborators

`PR Context Assembly`

#### Linked Requirements

`REQ-PRR-001`, `REQ-PRR-002`, `REQ-PRR-008`, `REQ-PRR-010`

### PR Context Assembly (FU-PR-CONTEXT-ASSEMBLY)

Fetches changed files and diff metadata and prepares bounded `review context` payloads.

#### Owned Responsibility

functional responsibility for `pr context assembly`; includes decision and `orchestration` `flow` to `GitHub App API` and decision and `orchestration` `flow` to `GitHub REST SDK`

#### Key Collaborators

`GitHub App API`, `GitHub REST SDK`, `Review Orchestration`

#### Linked Requirements

`REQ-PRR-002`

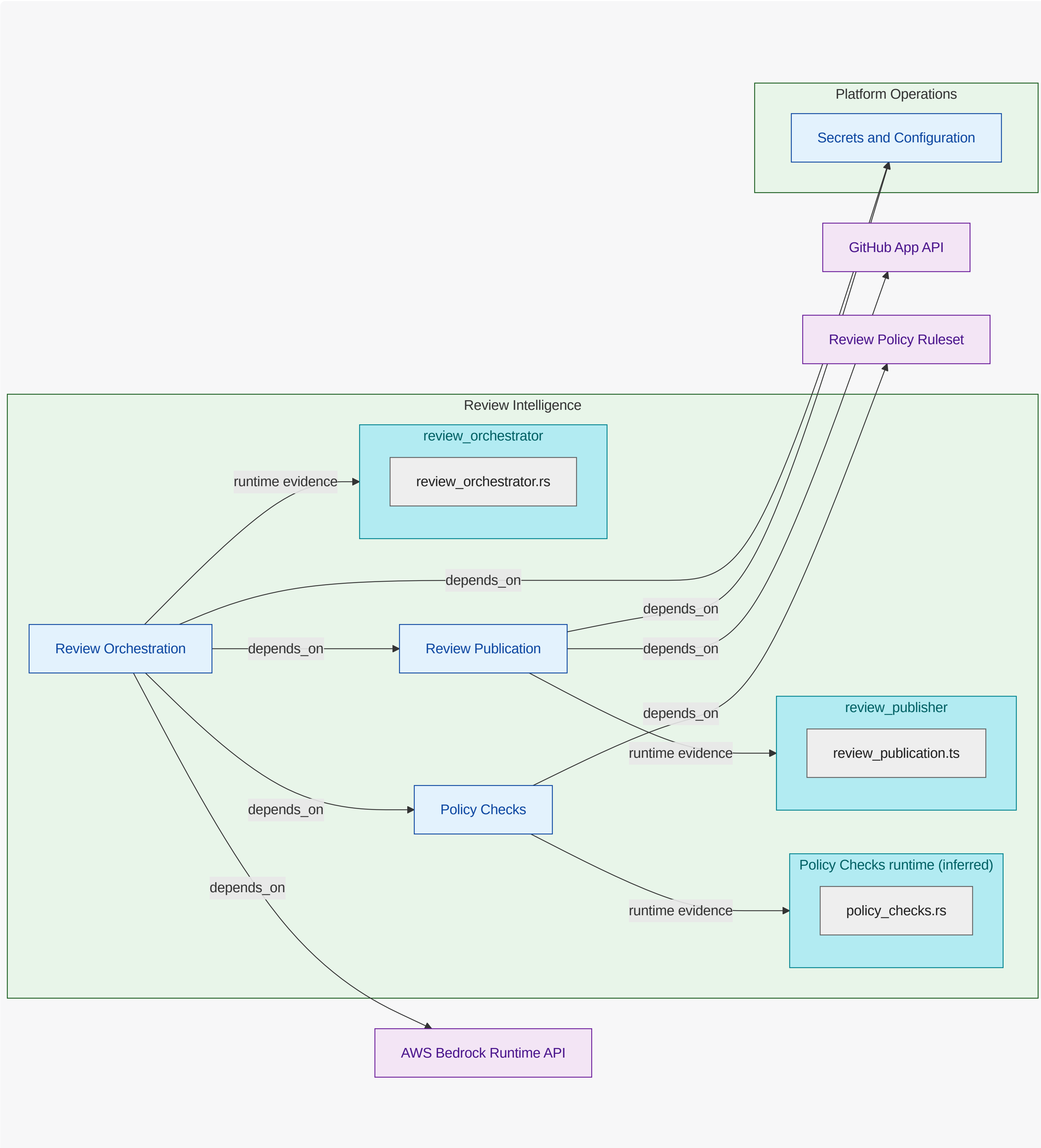
### 6.5.2. Review Intelligence (FG-REVIEW-INTELLIGENCE)

AI and deterministic review analysis with result `publication`.

```
    <<REF_IDX-CATALOG_FG_REVIEW_INTELLIGENCE,Review Intelligence>> owns generated findings, <<REF_IDX-
CATALOG_DETERMINISTIC_CHECKS,deterministic checks>>, and final <<REF_IDX-CATALOG_PUBLICATION,publication>> behavior.
It combines Bedrock reasoning with predictable <<REF_IDX-CATALOG_FU_POLICY_CHECKS,policy checks>> and emits auditable review
outputs.
```

#### Functional Unit Dependency Diagram

What this diagram shows: dependencies involving functional units in this functional group. Units in this group are rendered inside the group container; connected external units and referenced elements are shown outside it.




## Policy Checks (FU-POLICY-CHECKS)

Runs `deterministic checks` for secrets, risky IaC changes, and repository policy constraints.

### Owned Responsibility

functional responsibility for `policy checks`; includes decision and `orchestration` `flow` to Review `Policy Ruleset`

#### Key Collaborators

Review `Policy Ruleset`

#### Linked Requirements

`REQ-PRR-004`, `REQ-PRR-006`

### Review Orchestration (FU-REVIEW-ORCHESTRATION)

Calls Bedrock models, merges AI and deterministic findings, and handles graceful degradation paths.

#### Owned Responsibility

functional responsibility for `review orchestration`; includes decision and `orchestration` `flow` to AWS `Bedrock Runtime` API and decision and `orchestration` `flow` to `Policy Checks`

#### Key Collaborators

AWS `Bedrock Runtime` API, `Policy Checks`, `Review Publication`, `Secrets and Configuration`

#### Linked Requirements

`REQ-PRR-003`, `REQ-PRR-006`

### Review Publication (FU-REVIEW-PUBLICATION)

Publishes check-run summaries and inline comments back to GitHub pull requests.

#### Owned Responsibility

functional responsibility for `review publication`; includes decision and `orchestration` `flow` to `GitHub App API` and decision and `orchestration` `flow` to `Secrets and Configuration`

#### Key Collaborators

`GitHub App API`, `Secrets and Configuration`

#### Linked Requirements

`REQ-PRR-005`, `REQ-PRR-006`, `REQ-PRR-007`, `REQ-PRR-008`, `REQ-PRR-009`, `REQ-PRR-010`

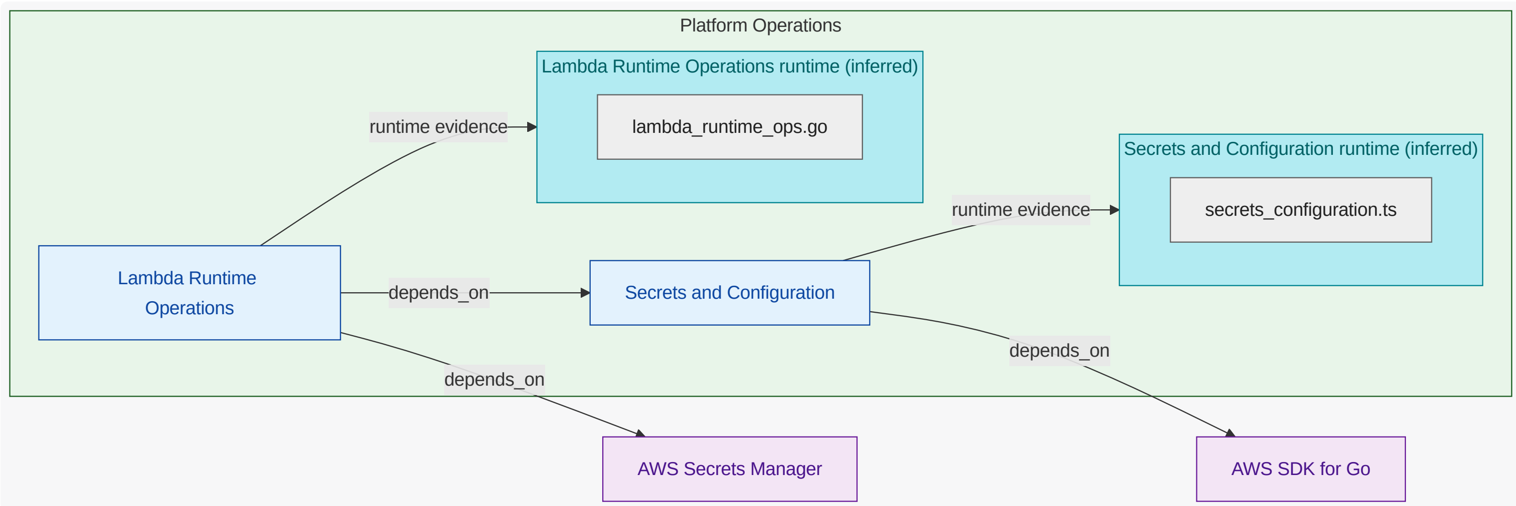
## 6.5.3. Platform Operations (FG-PLATFORM-OPERATIONS)

Lambda deployment lifecycle, `runtime operations`, and secrets/configuration.

```
    <<REF_IDX-CATALOG_FG_PLATFORM_OPERATIONS,Platform Operations>> owns safe deployment, runtime controls, and secret/config
lifecycle for the app.
It enables dependable review operation without owning business-level review policy semantics.
```

### Functional Unit Dependency Diagram

What this diagram shows: dependencies involving functional units in this functional group. Units in this group are rendered inside the group container; connected external units and referenced elements are shown outside it.




## Lambda Runtime Operations (FU-LAMBDA-RUNTIME-OPERATIONS)

Manages Lambda packaging/deployment lifecycle and operational health guardrails.

### Owned Responsibility

functional responsibility for `lambda runtime operations`; includes decision and `orchestration` `flow` to `AWS Secrets Manager` and decision and `orchestration` `flow` to `Secrets and Configuration`

### Key Collaborators

`AWS Secrets Manager`, `Secrets and Configuration`

### Linked Requirements

`REQ-PRR-011`

## Secrets and Configuration (FU-SECRETS-CONFIGURATION)

Manages GitHub App credentials, Bedrock config, and redaction-safe runtime configuration.

### Owned Responsibility

functional responsibility for `secrets and configuration`; includes decision and `orchestration` `flow` to `AWS SDK for Go`

### Key Collaborators

`AWS SDK for Go`

### Linked Requirements

`REQ-PRR-007`

# 7. Communication View

Show who communicates with whom across key request paths, including `interface` edges and dependency handoffs.

## 7.1. What This View Answers

- Who talks to whom and over which interfaces?
- Which edges are synchronous, asynchronous, event-driven, or callback-oriented?
- Which communication paths are critical to the main request `flow`?

## 7.2. Coverage Gaps

Coverage summary: 3/7 units have direct `evidence` coverage in this `view`.

- `Lambda Runtime Operations` has no direct inferred runtime/deployment `evidence` yet.
- `PR Context Assembly` has no direct inferred runtime/deployment `evidence` yet.
- `Policy Checks` has no direct inferred runtime/deployment `evidence` yet.
- `Secrets and Configuration` has no direct inferred runtime/deployment `evidence` yet.

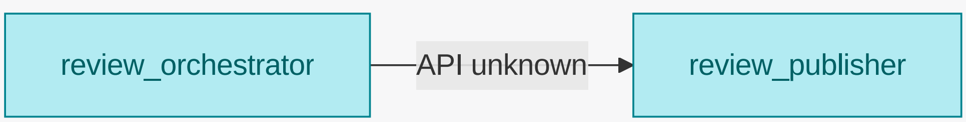
## 7.3. Recommended Next Evidence Additions

- Add explicit communication metadata (protocol, sync/async/`event`/callback) to `authored mappings` where currently implicit.
- Expose service protocol/port metadata in Helm or manifest values to enrich interface-edge inference.
- Capture missing communicating runtime units by adding discoverable workload artifacts under scanned inference roots.

## 7.4. Communication Path Graph

What this diagram shows: inferred unit-to-unit request/call flow, including available interface/port labels.

| NOTE | This graph is inferred from available deployment/runtime artifacts and may be partial. Missing edges or interface details indicate missing derived communication evidence, not necessarily missing behavior. |
|---|---|




## 7.5. Functional Groups and Units (View Scoped)

# 8. Deployment View

Show inferred deployment artifacts and ownership mapping to authored units. This `view` emphasizes deployment relationships and `platform operations`.

## 8.1. What This View Answers

- How does authored intent become deployed runtime?
- What control-plane objects manage deployable units?
- Where are platform ownership boundaries and missing ownership signals?

## 8.2. Coverage Gaps

Coverage summary: 3/7 units have direct `evidence` coverage in this `view`.

- `Lambda Runtime Operations` has no direct inferred runtime/deployment `evidence` yet.
- `PR Context Assembly` has no direct inferred runtime/deployment `evidence` yet.
- `Policy Checks` has no direct inferred runtime/deployment `evidence` yet.
- `Secrets and Configuration` has no direct inferred runtime/deployment `evidence` yet.

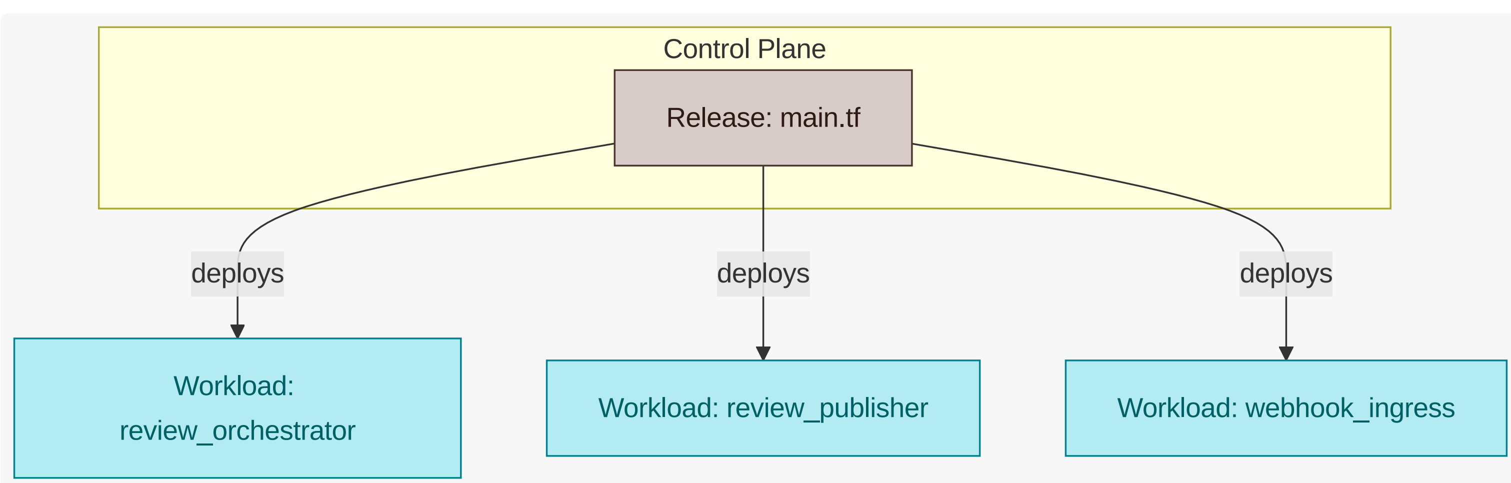
## 8.3. Recommended Next Evidence Additions

- Add owner metadata to Flux/Kustomization/Helm objects for clearer platform ownership mapping.
- Ensure deployment `control` artifacts (source, kustomization, release) are all included in scanned directories.
- Add deterministic naming conventions for release and namespace objects to improve cross-artifact linking.

## 8.4. Deployment Artifact Relationship Graph

What this diagram shows: how source/control-plane artifacts connect to releases, namespaces, workloads, and target cluster context.

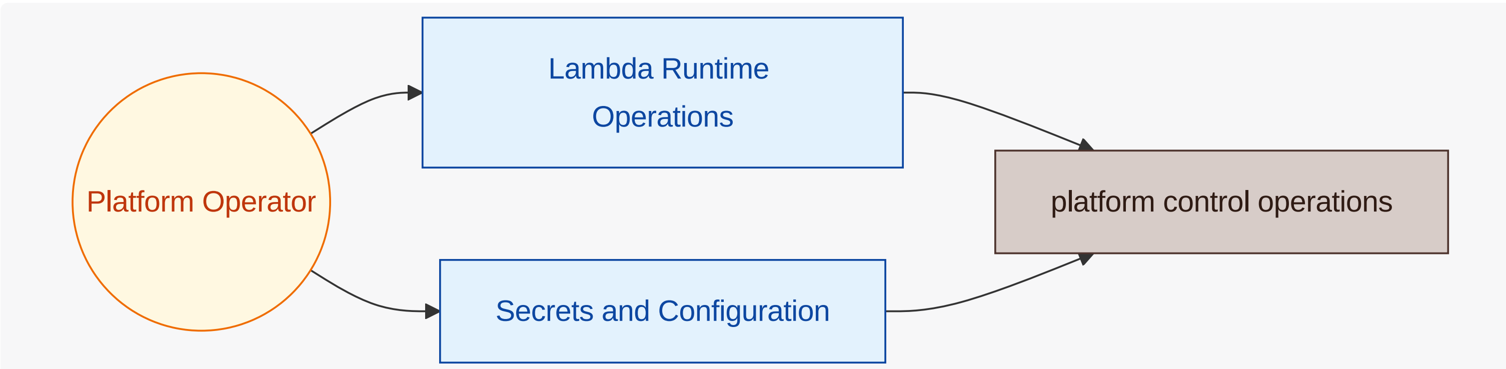
> **NOTE**
>
> This graph is inferred from available deployment artifacts (for example Terraform, Flux, Helm) and may be partial when source metadata is incomplete. Namespace and cluster boundaries are rendered as containers when target/placement relationships are inferred. Node labels include role hints (for example `Source`, `Kustomization`, `Release`, `Workload`, `ns/…`) to distinguish similarly named elements.



## 8.5. Platform Operations Graph

What this diagram shows: which platform actors operate which platform units and the control-plane targets they influence.

> **NOTE** Platform operations links are inferred from authored actor mappings plus discovered deployment control-plane artifacts.




## 8.6. Functional Groups and Units (View Scoped)

### 8.6.1. PR Integration (FG-PR-INTEGRATION)

`PR Integration` functions deploy independently to isolate `webhook ingress` from context-fetch implementation changes.

#### GitHub Webhook Ingress (FU-GITHUB-WEBHOOK-INGRESS)

##### GitHub Webhook Ingress Deployment

Deploys as an independently managed Lambda runtime boundary.

| Deployment Aspect | View-Scoped Detail |
|---|---|
| Deployment trigger context | `REQ-PRR-001`, `REQ-PRR-002`, `REQ-PRR-008`, `REQ-PRR-010` |
| Deployment dependencies | `PR Context Assembly` |
| Inferred deployment evidence | runtime: `webhook_ingress` |

#### PR Context Assembly (FU-PR-CONTEXT-ASSEMBLY)

##### PR Context Assembly Deployment

Deployed separately to allow context shaping changes without ingress disruption.

| Deployment Aspect | View-Scoped Detail |
|---|---|
| Deployment trigger context | `REQ-PRR-002` |
| Deployment dependencies | `GitHub App API`, `GitHub REST SDK`, `Review Orchestration` |
| Inferred deployment evidence | code modules: `pr_context_assembly` |

### 8.6.2. Review Intelligence (FG-REVIEW-INTELLIGENCE)

`Review Intelligence` units roll independently to preserve availability and reduce blast radius during `model` or rule updates.

#### Policy Checks (FU-POLICY-CHECKS)

#### Policy Checks Deployment

Deployed independently to evolve policy logic without `model` coupling.

| Deployment Aspect | View-Scoped Detail |
|---|---|
| Deployment trigger context | `REQ-PRR-004`, `REQ-PRR-006` |
| Deployment dependencies | Review `Policy Ruleset` |
| Inferred deployment evidence | code modules: `policy_checks` |

### Review Orchestration (FU-REVIEW-ORCHESTRATION)

#### Review Orchestration Deployment

Isolated deployment enables cautious `model`/config rollout and fallback `control`.

| Deployment Aspect | View-Scoped Detail |
|---|---|
| Deployment trigger context | `REQ-PRR-003`, `REQ-PRR-006` |
| Deployment dependencies | AWS `Bedrock Runtime` API, `Policy Checks`, `Review Publication`, `Secrets and Configuration` |
| Inferred deployment evidence | runtime: `review_orchestrator` |

### Review Publication (FU-REVIEW-PUBLICATION)

#### Review Publication Deployment

Release isolation protects `publication` reliability during other unit rollouts.

| Deployment Aspect | View-Scoped Detail |
|---|---|
| Deployment trigger context | `REQ-PRR-005`, `REQ-PRR-006`, `REQ-PRR-007`, `REQ-PRR-008`, `REQ-PRR-009`, `REQ-PRR-010` |
| Deployment dependencies | `GitHub App API`, `Secrets and Configuration` |
| Inferred deployment evidence | runtime: `review_publisher` |

### 8.6.3. Platform Operations (FG-PLATFORM-OPERATIONS)

`Platform Operations` governs release `orchestration` and runtime placement to keep review capacity stable.

### Lambda Runtime Operations (FU-LAMBDA-RUNTIME-OPERATIONS)

#### Lambda Runtime Operations Deployment

Governs source-to-release pipeline behavior and runtime placement boundaries.

| Deployment Aspect | View-Scoped Detail |
|---|---|
| Deployment trigger context | `REQ-PRR-011` |
| Deployment dependencies | `AWS Secrets Manager`, `Secrets and Configuration` |
| Inferred deployment evidence | code modules: `lambda_runtime_ops` |

### Secrets and Configuration (FU-SECRETS-CONFIGURATION)

### Secrets and Configuration Deployment

Managed as a shared operational boundary with controlled rotation and rollout `flow`.

| Deployment Aspect | View-Scoped Detail |
| --- | --- |
| Deployment trigger context | `REQ-PRR-007` |
| Deployment dependencies | `AWS SDK for Go` |
| Inferred deployment evidence | code modules: `secrets_configuration` |

# 9. Security View

Show inferred exposure and dependency `risk` points aligned to unit boundaries, focused on attack paths and security `evidence`.

## 9.1. What This View Answers

- What are the main `trust boundaries` and attack paths?
- Which units are exposed and what security `evidence` exists?
- Where is the threat `model` incomplete?

## 9.2. Coverage Gaps

Coverage summary: 5/7 units have direct `evidence` coverage in this `view`.

- `Lambda Runtime Operations` has no explicit authored attack-vector mapping yet; document technical and fraud/abuse threats even when mitigated.
- `Policy Checks` has no explicit authored attack-vector mapping yet; document technical and fraud/abuse threats even when mitigated.

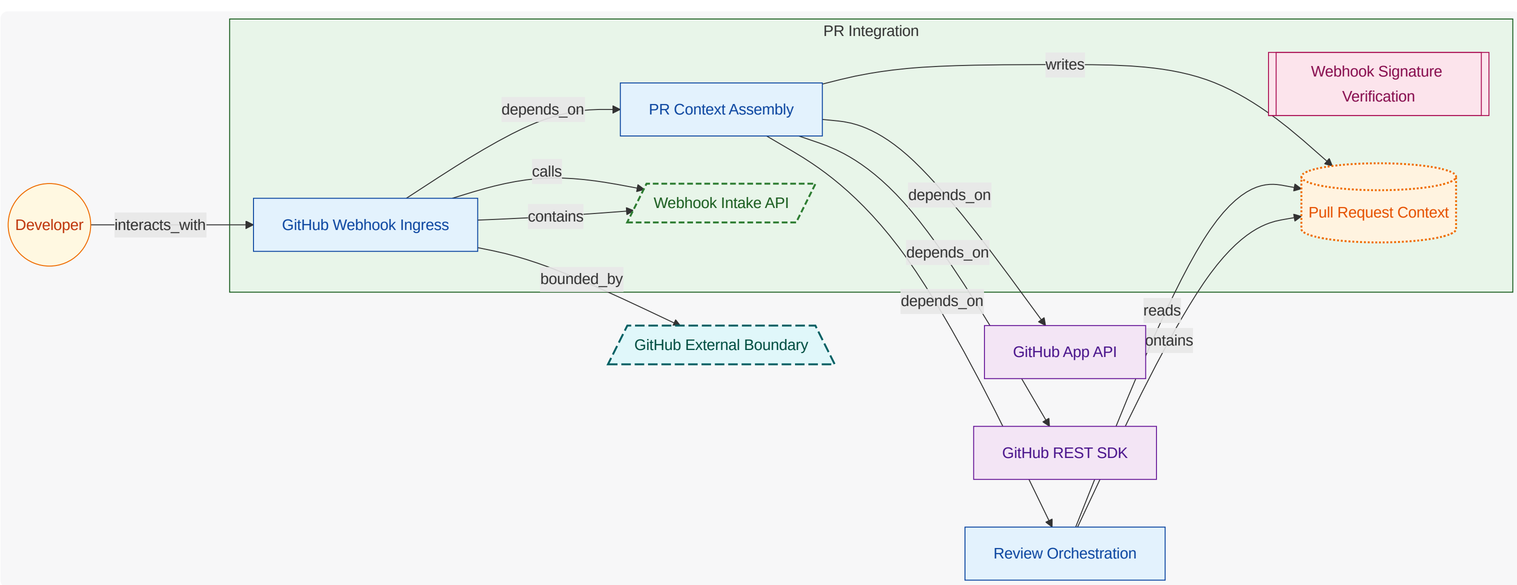
## 9.3. Recommended Next Evidence Additions

- Add explicit authored attack-vector mappings for units currently marked without threat linkage.
- Add security signal ownership metadata (logs/alerts/`events`) for unresolved observability `evidence`.
- Map additional security-relevant dependencies and exposures into `authored mappings` for traceable coverage.

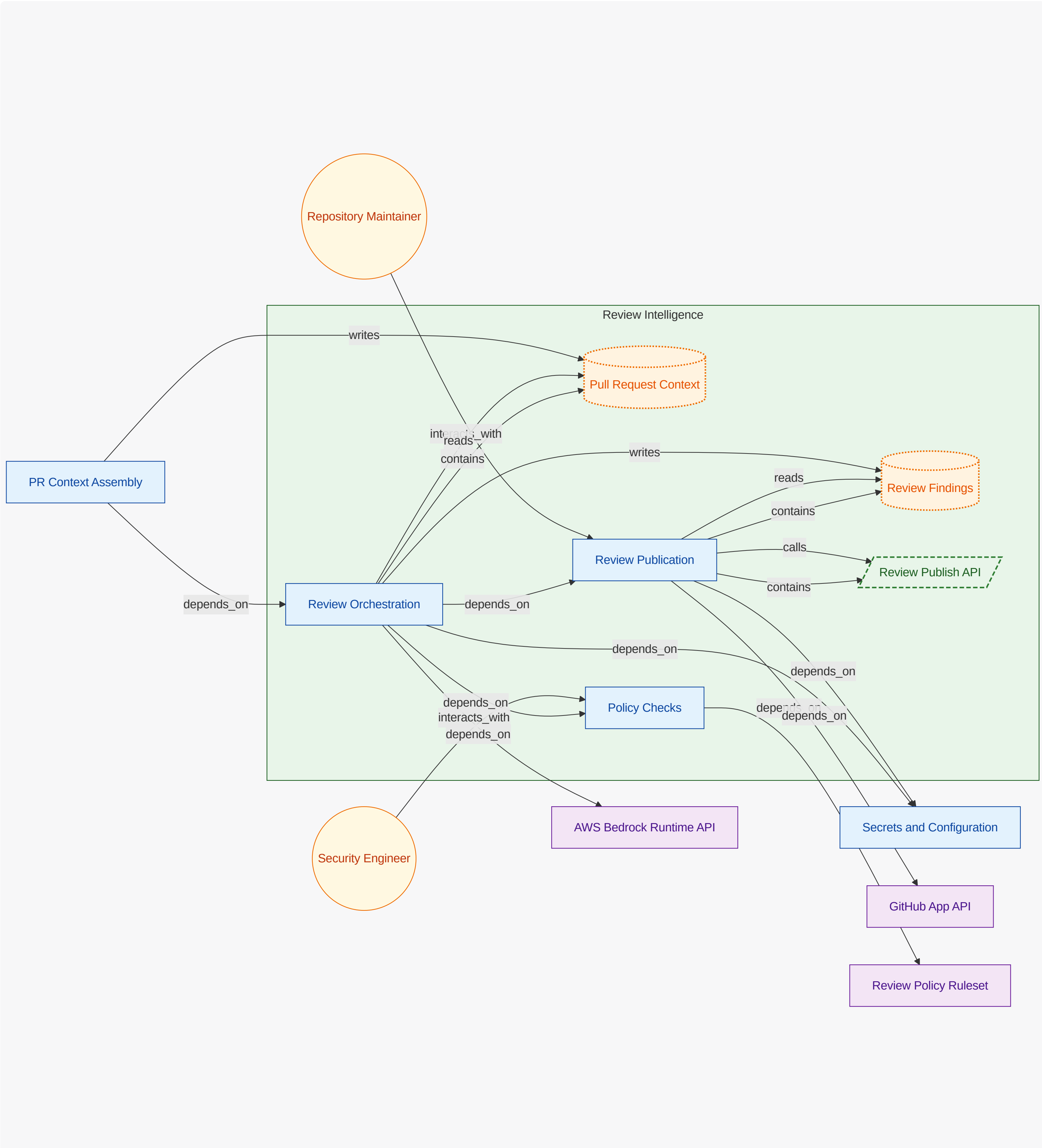
## 9.4. Threat Model Context Diagram

What these diagrams show: OWASP-style context views split by functional group. Each group diagram keeps owned functional units and owned security-relevant entities inside the functional-group container; connected actors, referenced elements, external functional units, and other external entities are rendered outside it.

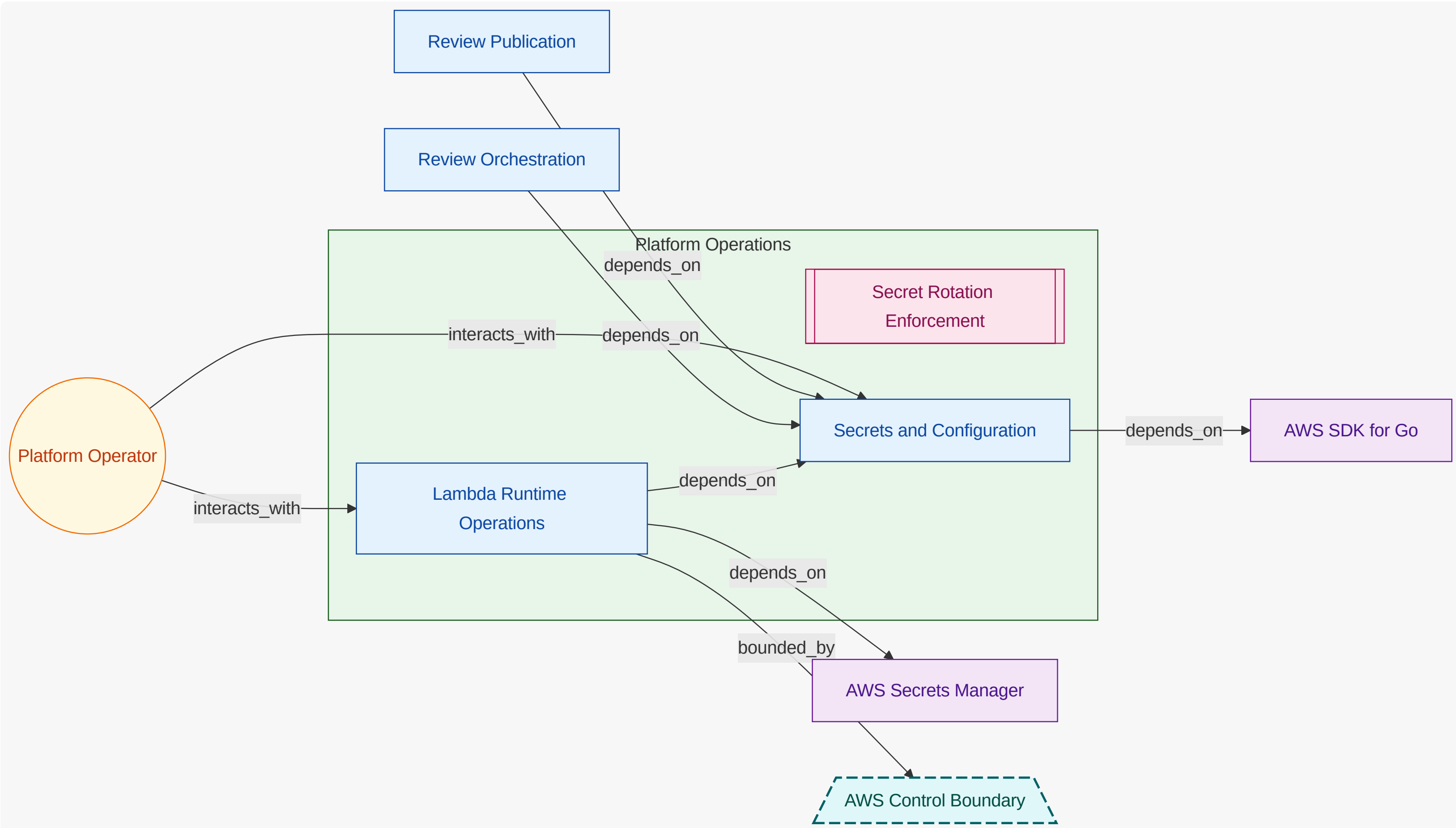
### 9.4.1. PR Integration (FG-PR-INTEGRATION)




### 9.4.2. Review Intelligence (FG-REVIEW-INTELLIGENCE)




## 9.4.3. Platform Operations (FG-PLATFORM-OPERATIONS)




## 9.5. Threat Scenario Register

### 9.5.1. `Prompt injection in diff` influences review output integrity (TS-BEDROCK-PROMPT-INJECTION)

Malicious prompts in diff content attempt to coerce `model` output away from policy intent.

| Field | Value |
|---|---|
| Status | mitigating |
| Owner | `Security Engineer` |
| Attack Vector | `Prompt Injection in Diff` |
| Scope | `PR Context Assembly`, `Pull Request Context`, `Review Orchestration` |
| Flows | `PR Review Interaction Flow` |
| Likelihood | medium |
| Impact | medium |
| Severity | medium |
| Risk | RISK-BEDROCK-PROMPT-INJECTION |
| Controls | `Webhook Signature Verification` |
| Mitigations | MIT-BEDROCK-CONTEXT-SANITIZATION |

| Field | Value |
|---|---|
| Verifications | CV-BEDROCK-PROMPT-INJECTION-RESILIENCE |
| Operational Notes | Track closure through mitigation and verification linkage. |

#### Control Verification Status

##### CV-BEDROCK-PROMPT-INJECTION-RESILIENCE

| Field | Value |
|---|---|
| Control | Webhook Signature Verification |
| Method | analysis |
| Status | partial |
| Owner | Security Engineer |
| Last Tested | 2026-04-19 |
| Risks | RISK-BEDROCK-PROMPT-INJECTION |
| Findings | deterministic policy merge blocks obvious prompt directives; edge cases remain under review |

### 9.5.2. Sensitive tokens leak via review publication payload (TS-BEDROCK-SECRET-LEAK-IN-PUBLICATION)

Review output includes sensitive content due to incomplete redaction before publication.

| Field | Value |
|---|---|
| Status | mitigating |
| Owner | Security Engineer |
| Attack Vector | Secret Leakage in Review Output |
| Scope | Review Findings, Review Publication, Secrets and Configuration |
| Flows | PR Review Interaction Flow, Security Policy Tuning Flow |
| Likelihood | low |
| Impact | high |
| Severity | medium |
| Risk | RISK-BEDROCK-SECRET-LEAKAGE |
| Controls | Secret Rotation Enforcement |
| Mitigations | MIT-BEDROCK-OUTPUT-REDACTION |
| Verifications | CV-BEDROCK-REDACTION-CHECK |

| Field | Value |
|---|---|
| Operational Notes | Track closure through mitigation and verification linkage. |

#### Control Verification Status

##### CV-BEDROCK-REDACTION-CHECK

| Field | Value |
|---|---|
| Control | Secret Rotation Enforcement |
| Method | inspection |
| Status | partial |
| Owner | Security Engineer |
| Last Tested | 2026-04-21 |
| Risks | RISK-BEDROCK-SECRET-LEAKAGE |
| Findings | redaction filters exist; custom token patterns are still being expanded |

### 9.5.3. Spoofed webhook bypasses ingress verification (TS-BEDROCK-WEBHOOK-SPOOF)

Forged pull request webhook attempts to trigger unauthorized review execution.

| Field | Value |
|---|---|
| Status | mitigating |
| Owner | Security Engineer |
| Attack Vector | Spoofed GitHub Webhook |
| Scope | GitHub Webhook Ingress , Webhook Intake API |
| Flows | PR Review Interaction Flow |
| Likelihood | medium |
| Impact | high |
| Severity | high |
| Risk | RISK-BEDROCK-WEBHOOK-SPOOFING |
| Controls | Webhook Signature Verification |
| Mitigations | MIT-BEDROCK-WEBHOOK-HMAC |
| Verifications | CV-BEDROCK-WEBHOOK-SIGNATURE |
| Operational Notes | Track closure through mitigation and verification linkage. |

#### Control Verification Status

CV-BEDROCK-WEBHOOK-SIGNATURE

| Field | Value |
|---|---|
| Control | Webhook Signature Verification |
| Method | test |
| Status | pass |
| Owner | Security Engineer |
| Last Tested | 2026-04-11 |
| Risks | RISK-BEDROCK-WEBHOOK-SPOOFING |
| Findings | invalid webhook signatures rejected before context assembly |

## 9.6. Threat Mitigation Coverage

| ID | Threat Scenario | Control | Status | Effectiveness | Owner | Verification Links | Notes |
|---|---|---|---|---|---|---|---|
| MIT-BEDROCK-CONTEXT-SANITIZATION | TS-BEDROCK-PROMPT-INJECTION | Webhook Signature Verification | partial | medium | Security Engineer | CV-BEDROCK-PROMPT-INJECTION-RESILIENCE | Context sanitizer and deterministic policy checks reduce injection impact; coverage expansion continues. |
| MIT-BEDROCK-OUTPUT-REDACTION | TS-BEDROCK-SECRET-LEAK-IN-PUBLICATION | Secret Rotation Enforcement | partial | medium | Security Engineer | CV-BEDROCK-REDACTION-CHECK | Publication redaction pipeline blocks common secret shapes and is being extended. |
| MIT-BEDROCK-WEBHOOK-HMAC | TS-BEDROCK-WEBHOOK-SPOOF | Webhook Signature Verification | verified | high | Security Engineer | CV-BEDROCK-WEBHOOK-SIGNATURE | HMAC signature checks and ingress rejection reduce spoofing attack viability. |

## 9.7. Security Flow Semantics

| ID | Title | Kind | Direction | Frequency | Source | Destination | Protocol |
|---|---|---|---|---|---|---|---|
| FLOW-BEDROCK-POLICY-TUNING | Security Policy Tuning Flow | control | internal | on-demand | Security Engineer | Review Publication | internal-api |

| ID | Title | Kind | Direction | Frequency | Source | Destination | Protocol |
| --- | --- | --- | --- | --- | --- | --- | --- |
| FLOW-BEDROCK-PR-REVIEW | PR Review Interaction Flow | security | inbound | realtime | Webhook Intake API | Review Publication | https |

| ID | Auth | Encryption | Integrity | Boundary Crossing | Trust Boundary | Threats | Data |
| --- | --- | --- | --- | --- | --- | --- | --- |
| FLOW-BEDROCK-POLICY-TUNING | workforce-sso | tls1.3 | signed-policy-bundle | false | none | TS-BEDROCK-SECRET-LEAK-IN-PUBLICATION | Pull Request Context , Review Findings |
| FLOW-BEDROCK-PR-REVIEW | hmac-signature | tls1.3 | signed-payload | true | AWS Control Boundary | TS-BEDROCK-PROMPT-INJECTION, TS-BEDROCK-WEBHOOK-SPOOF | Pull Request Context , Review Findings |

## 9.8. Threat Assumptions

| ID | Title | Status | Owner | Applies To | Rationale |
| --- | --- | --- | --- | --- | --- |
| ASM-BEDROCK-GITHUB-SIGNATURE-TRUST | GitHub webhook signature secret remains uncompromised | accepted | Security Engineer | AWS Secrets Manager , GitHub Webhook Ingress , Webhook Intake API | Secret rotation and access controls are centrally governed by platform policy. |

## 9.9. Threat Model Out-of-Scope Decisions

| ID | Title | Status | Owner | Applies To | Expires On | Reason |
| --- | --- | --- | --- | --- | --- | --- |
| OOS-BEDROCK-GITHUB-CLOUD-PLATFORM | GitHub SaaS internal tenant isolation controls | approved | Security Engineer | GitHub App API , GitHub Integration Boundary | 2027-12-31 | Internal platform isolation and abuse controls for GitHub SaaS are external to this system boundary. |

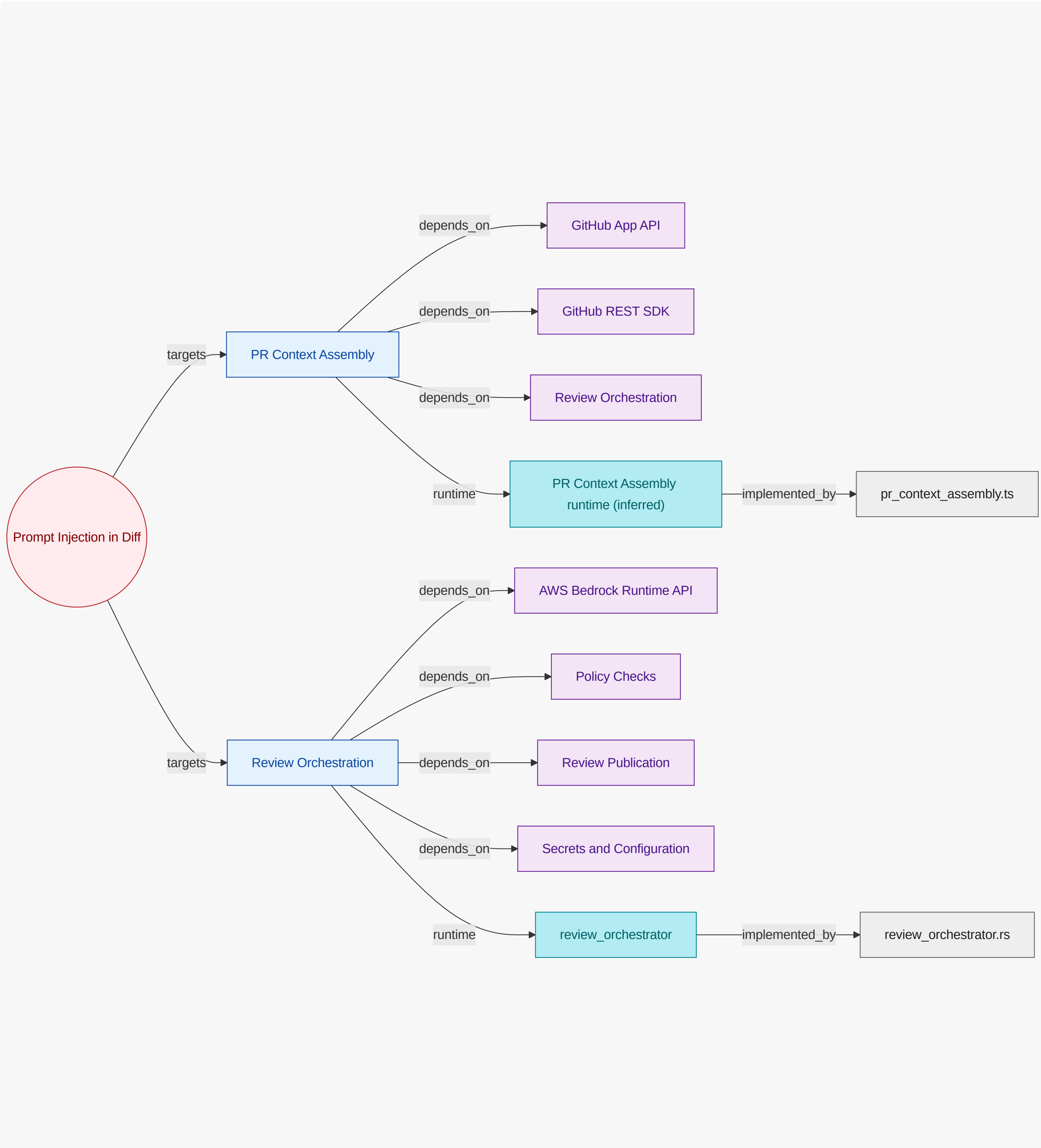
## 9.10. Attack Vector Chapters

### 9.10.1. Prompt Injection in Diff (AV-PROMPT-INJECTION-IN-DIFF)

Malicious prompt-like content embedded in pull request diffs.

Mitigated by: none

Trust boundaries: none




### PR Context Assembly (FU-PR-CONTEXT-ASSEMBLY)

Applies contextual sanitization to reduce `prompt injection` `risk` from diffs.

#### Attack Surface

`GitHub App API`, `GitHub REST SDK`, `Review Orchestration`

#### Security Evidence Present

code modules: pr_context_assembly

### Review Orchestration (FU-REVIEW-ORCHESTRATION)

Enforces safe prompt construction and bounded output handling under adversarial input.

#### Attack Surface

AWS Bedrock Runtime API, Policy Checks, Review Publication, Secrets and Configuration

#### Security Evidence Present

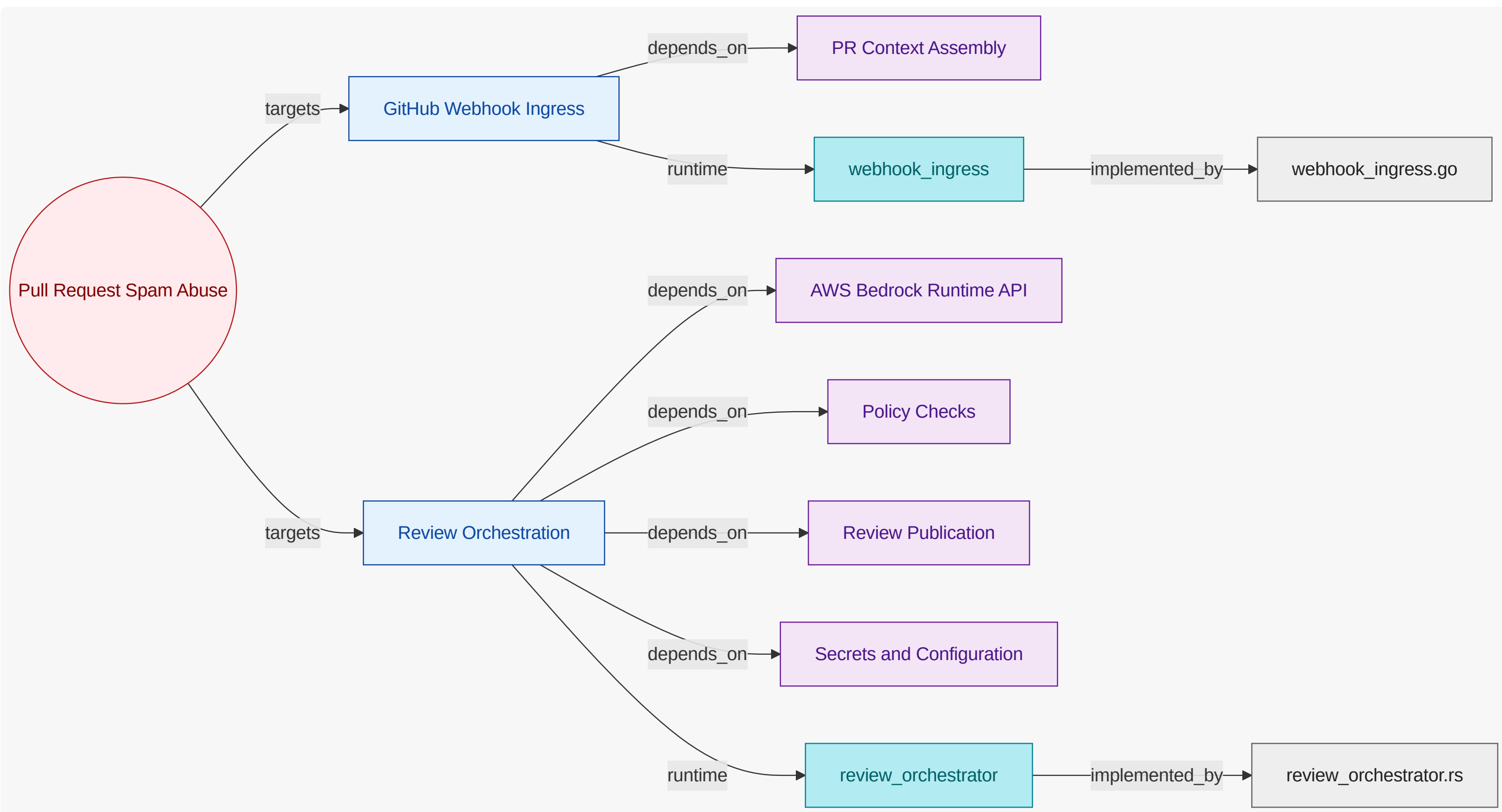
runtime: review_orchestrator | code modules: review_orchestrator

## 9.10.2. Pull Request Spam Abuse (AV-PR-SPAM-ABUSE)

High-rate PR event flooding that can exhaust review capacity.

Mitigated by: none

Trust boundaries: GitHub External Boundary




### GitHub Webhook Ingress (FU-GITHUB-WEBHOOK-INGRESS)

Enforces signature verification and rejects spoofed or malformed events.

#### Attack Surface

PR Context Assembly

#### Security Evidence Present

runtime: webhook_ingress | code modules: webhook_ingress

### Review Orchestration (FU-REVIEW-ORCHESTRATION)

Enforces safe prompt construction and bounded output handling under adversarial input.

#### Attack Surface

AWS Bedrock Runtime API, Policy Checks, Review Publication, Secrets and Configuration

#### Security Evidence Present

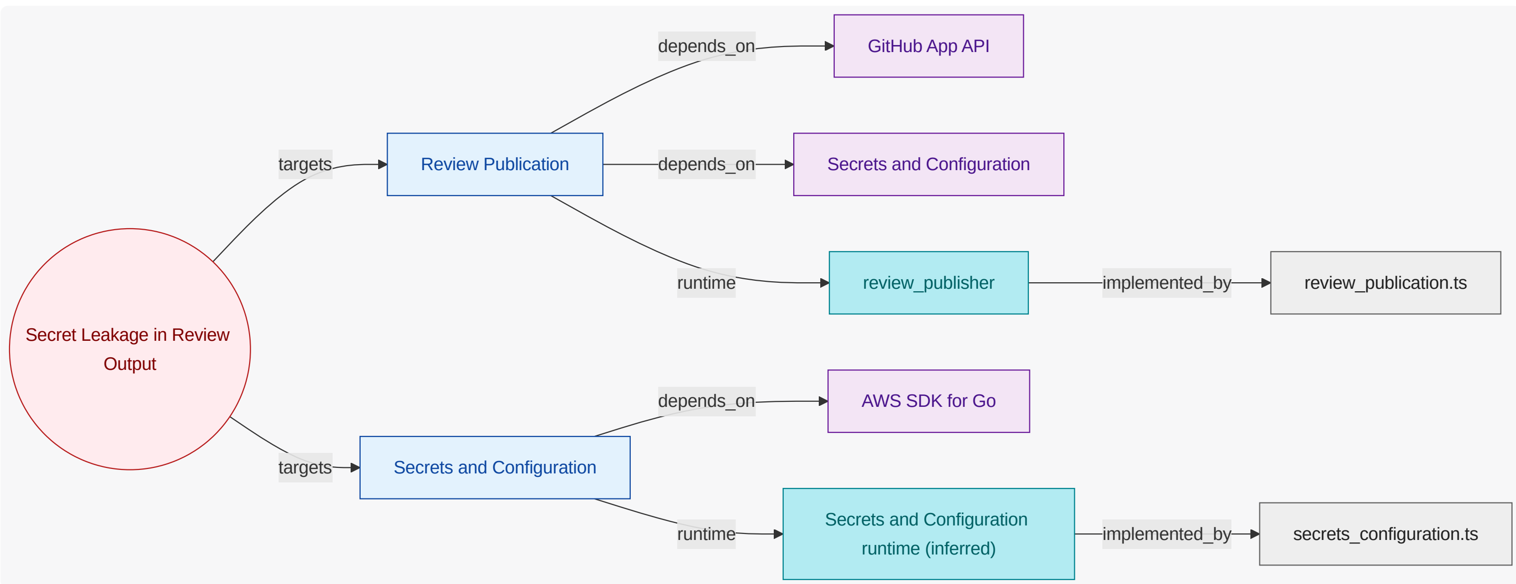
runtime: review_orchestrator | code modules: review_orchestrator

### 9.10.3. Secret Leakage in Review Output (AV-SECRET-LEAKAGE-IN-REVIEW)

Generated findings accidentally exposing secrets/tokens in comments.

Mitigated by: Secret Rotation Enforcement

Trust boundaries: none




#### Review Publication (FU-REVIEW-PUBLICATION)

Applies redaction and policy-aware output filtering before publishing findings.

#### Attack Surface

GitHub App API, Secrets and Configuration

#### Security Evidence Present

runtime: review_publisher | code modules: review_publication

#### Secrets and Configuration (FU-SECRETS-CONFIGURATION)

Enforces credential isolation, least-privilege access, and redaction-safe metadata handling.

#### Attack Surface

AWS SDK for Go

#### Security Evidence Present

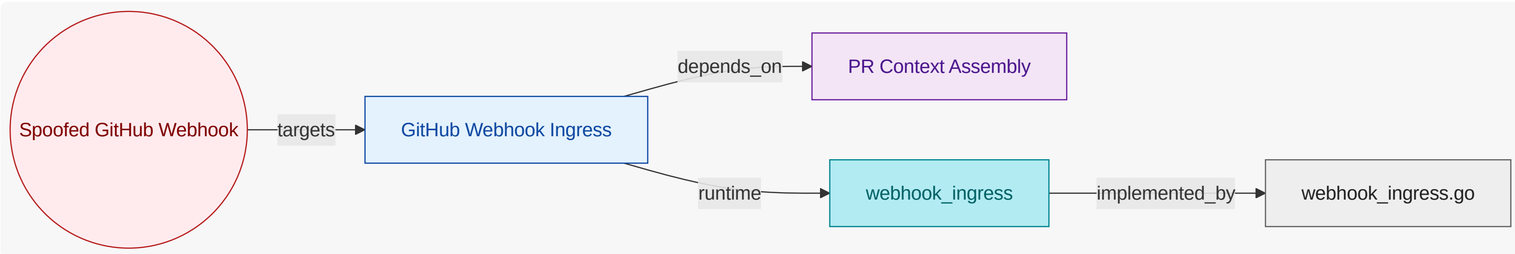
code modules: secrets_configuration

### 9.10.4. Spoofed GitHub Webhook (AV-SPOOFED-GITHUB-WEBHOOK)

Forged webhook payload attempting unauthorized review execution.

Mitigated by: Webhook Signature Verification

Trust boundaries: GitHub External Boundary




### GitHub Webhook Ingress (FU-GITHUB-WEBHOOK-INGRESS)

Enforces signature verification and rejects spoofed or malformed `events`.

#### Attack Surface

`PR Context Assembly`

#### Security Evidence Present

runtime: `webhook_ingress` | code modules: `webhook_ingress`

# 10. Traceability View

Show requirement-to-unit-to-evidence traceability, including coverage confidence and explicit `evidence` gaps.

## 10.1. What This View Answers

- Which `requirements` map to which units?
- Which authored units have runtime/code `evidence` and which do not?
- Where are confidence or completeness gaps in trace coverage?

## 10.2. Coverage Gaps

Coverage summary: 7/7 units have direct `evidence` coverage in this `view`.

- `Lambda Runtime Operations` has no direct inferred runtime `evidence` yet.
- `PR Context Assembly` has no direct inferred runtime `evidence` yet.
- `Policy Checks` has no direct inferred runtime `evidence` yet.
- `Secrets and Configuration` has no direct inferred runtime `evidence` yet.

## 10.3. Recommended Next Evidence Additions

- Add explicit requirement-to-owner mappings where `requirement` scope is currently broad or ambiguous.
- Add ownership annotations for runtime and code artifacts where coverage is still unresolved.
- Prioritize closing gaps for units that are authored but lack both runtime and code `evidence`.

# 11. State Lifecycle View

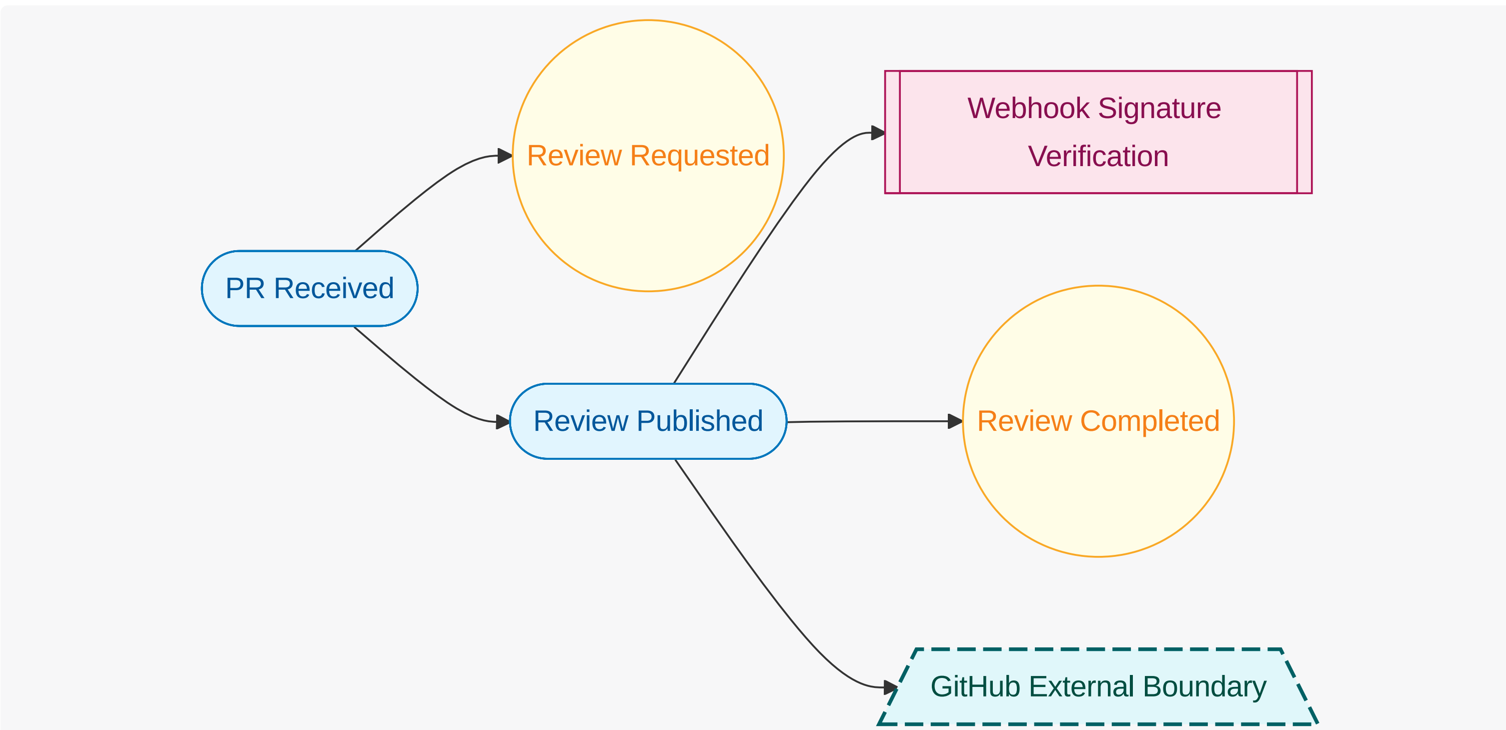
Show workflow `state` transitions, ownership of transitions, and key exceptional paths.

## 11.1. What This View Answers

- What `states` exist and which `events` trigger transitions?
- Which unit owns each `state` transition?
- Where do automatic and manual flows diverge?

## 11.2. State Lifecycle Diagram

What this diagram shows: authored lifecycle states, triggering events, guard controls, trust boundaries, and lifecycle transitions for this view.




## 11.3. Coverage Gaps

Coverage summary: No `functional units` are in scope for this `view`.

- No major coverage gaps detected from current authored and inferred inputs.

## 11.4. Recommended Next Evidence Additions

- No immediate `evidence` additions are required for this `view`.

## 11.5. Functional Groups and Units (View Scoped)

# 12. Interaction Flow View

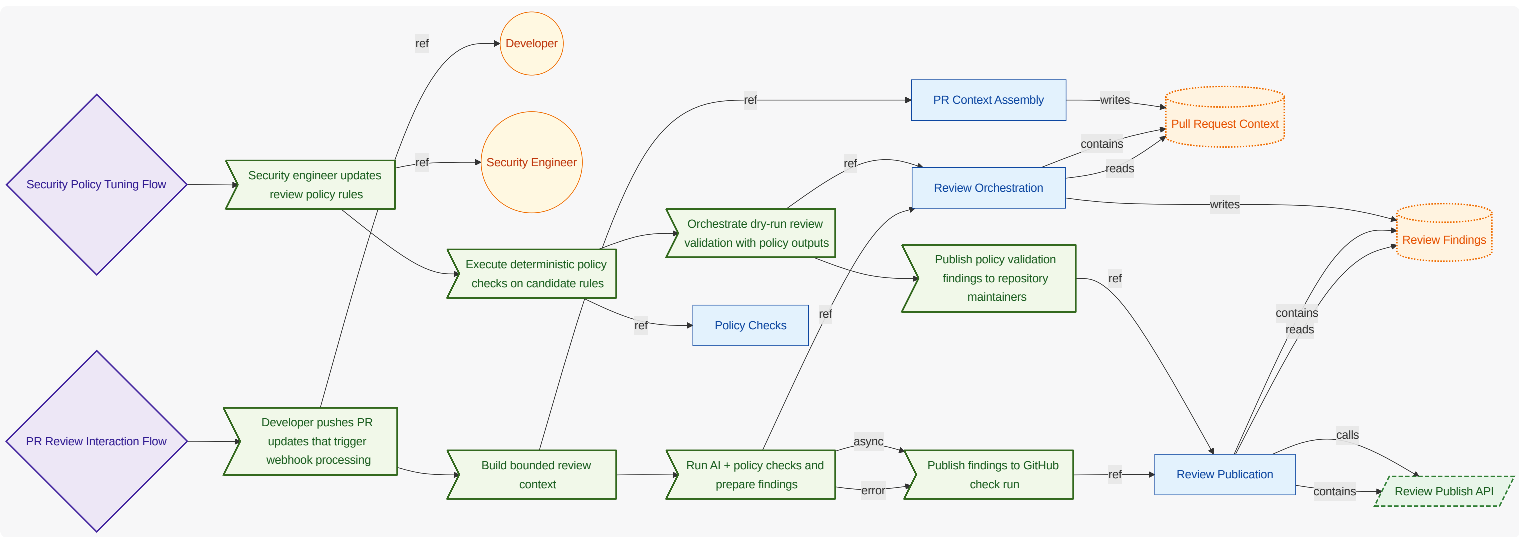
Show user-intent-to-outcome causal `flow`, including ordered steps, data movement, and error/async branches.

## 12.1. What This View Answers

- What happens from user intent to system outcome, step by step?
- What data enters/exits each step and where are decisions made?
- Where are async and error paths and what completes each path?

## 12.2. Interaction Flow Diagram

What this diagram shows: authored interaction flow steps, step ordering, async markers, and referenced actors/interfaces/units for this view.




## 12.3. Coverage Gaps

Coverage summary: 4/4 units have direct `evidence` coverage in this `view`.

- No major coverage gaps detected from current authored and inferred inputs.

## 12.4. Recommended Next Evidence Additions

- No immediate `evidence` additions are required for this `view`.

## 12.5. Functional Groups and Units (View Scoped)

## 12.6. Requirement-to-Unit Mapping (Compact)

What this matrix shows: authored requirement-to-unit mappings in a compact square layout (requirements as rows, functional units as columns).

| Requirement | FU-GITHUB-WEBHOOK-INGRESS | FU-LAMBDA-RUNTIME-OPERATIONS | FU-POLICY-CHECKS | FU-PR-CONTEXT-ASSEMBLY | FU-REVIEW-ORCHESTRATION | FU-REVIEW-PUBLICATION | FU-SECRETS-CONFIGURATION |
|---|---|---|---|---|---|---|---|
| REQ-PRR-001 | X | | | | | | |
| REQ-PRR-002 | X | | | X | | | |
| REQ-PRR-003 | | | | | X | | |
| REQ-PRR-004 | | | X | | | | |
| REQ-PRR-005 | | | | | | X | |
| REQ-PRR-006 | | | X | | X | X | |
| REQ-PRR-007 | | | | | | X | X |
| REQ-PRR-008 | X | | | | | X | |
| REQ-PRR-009 | | | | | | X | |
| REQ-PRR-010 | X | | | | | X | |
| REQ-PRR-011 | | X | | | | | |

## 12.7. Verification Result Mapping

What this section shows: per-requirement result mapping inferred from test-result artifacts.

| Check ID | Requirement | Result | Evidence / Notes |
|---|---|---|---|
| `VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5` | `REQ-PRR-001` | pass | test-results/e2e-pr-opened-review.xml |
| `VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5` | `REQ-PRR-002` | pass | test-results/e2e-pr-opened-review.xml |
| `VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5` | `REQ-PRR-003` | pass | test-results/e2e-pr-opened-review.xml |
| `VER-INF-INTEGRATION-POLICY_ONLY_FALLBACK_TEST-76E20A` | `REQ-PRR-004` | pass | test-results/integration-policy-fallback.json; deterministic `policy checks` executed |
| `VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5` | `REQ-PRR-005` | pass | test-results/contract-checkrun.json; check run payload includes summary and neutral conclusion |
| `VER-INF-TEST-GITHUB_CHECK_RUN_CONTRACT_TEST-17DBA6` | `REQ-PRR-005` | not-run | none; No parsed test-result artifact for this `requirement` yet. |
| `VER-INF-INTEGRATION-POLICY_ONLY_FALLBACK_TEST-76E20A` | `REQ-PRR-006` | pass | test-results/integration-policy-fallback.json; policy-only fallback published |

| Check ID | Requirement | Result | Evidence / Notes |
| --- | --- | --- | --- |
| VER-INF-INTEGRATION-AUDIT_REDACTION_TEST-B108A1 | REQ-PRR-007 | not-run | none; No parsed test-result artifact for this requirement yet. |
| VER-INF-UNIT-RATE_LIMIT_DEFERRAL_TEST-DC9EB2 | REQ-PRR-008 | pass | test-results/unit-rate-limit.xml |

# 13. Traceability Appendix

## 13.1. Requirement Details

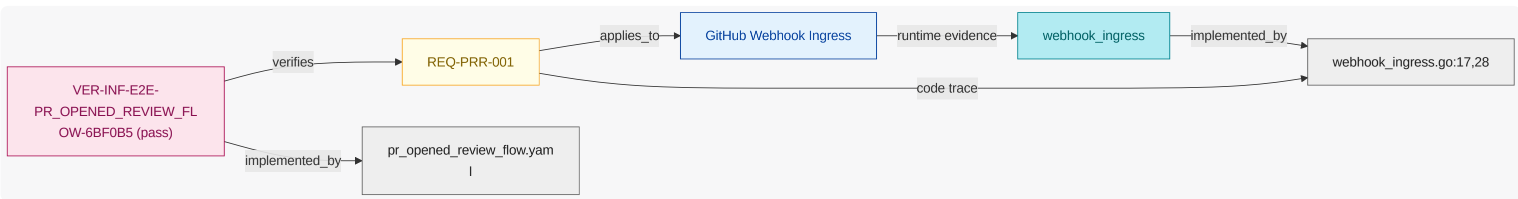
### 13.1.1. REQ-PRR-001

While normal mode is active, when pull request synchronize event is received and webhook signature is valid, the pr review system shall verify webhook signature before processing review workflow.

**NOTE** Webhook trust boundary validation.

#### Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



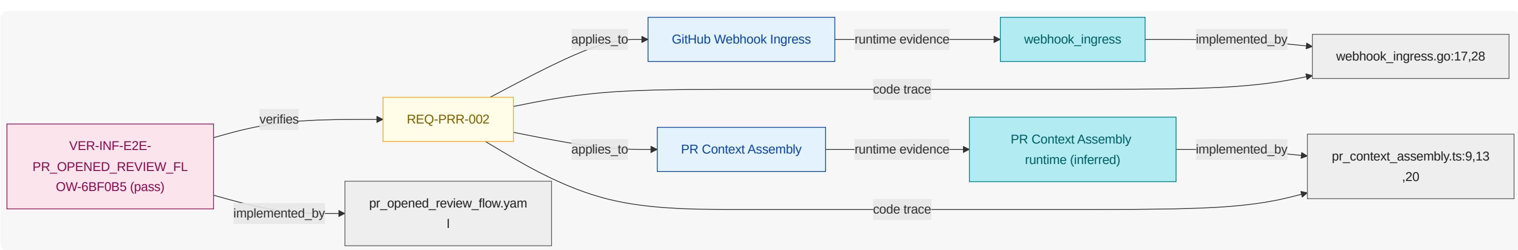

### 13.1.2. REQ-PRR-002

While pull request review is active, when pull request opened event is received, the pr review system shall construct diff context from changed files for developer review execution.

**NOTE** Context ingestion from GitHub API.

#### Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



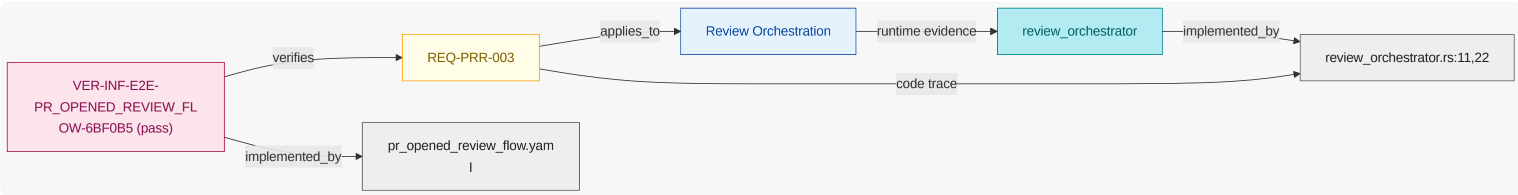

### 13.1.3. REQ-PRR-003

Where ai review is enabled, when diff context is prepared, the pr review system shall request bedrock runtime analysis for review findings.

**NOTE** AI-assisted review path.

#### Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



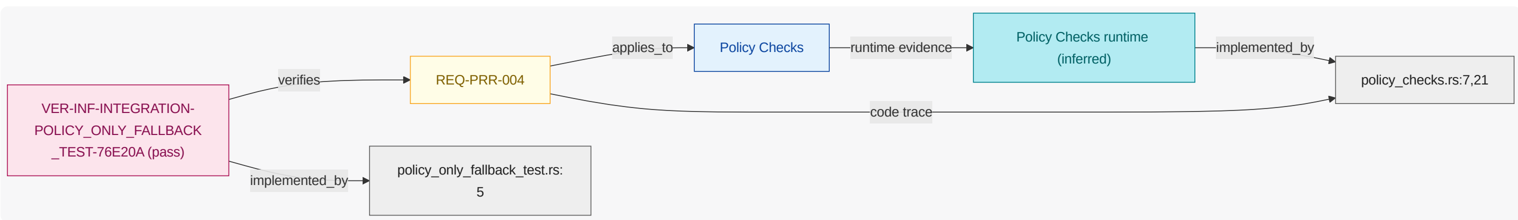

### 13.1.4. REQ-PRR-004

Where `policy checks are enabled`, when `pull request review is active`, `the pr review system` shall evaluate deterministic `policy checks` for secret exposure plus infrastructure `risk` plus dependency changes.

> **NOTE** Deterministic non-LLM review checks.

#### Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



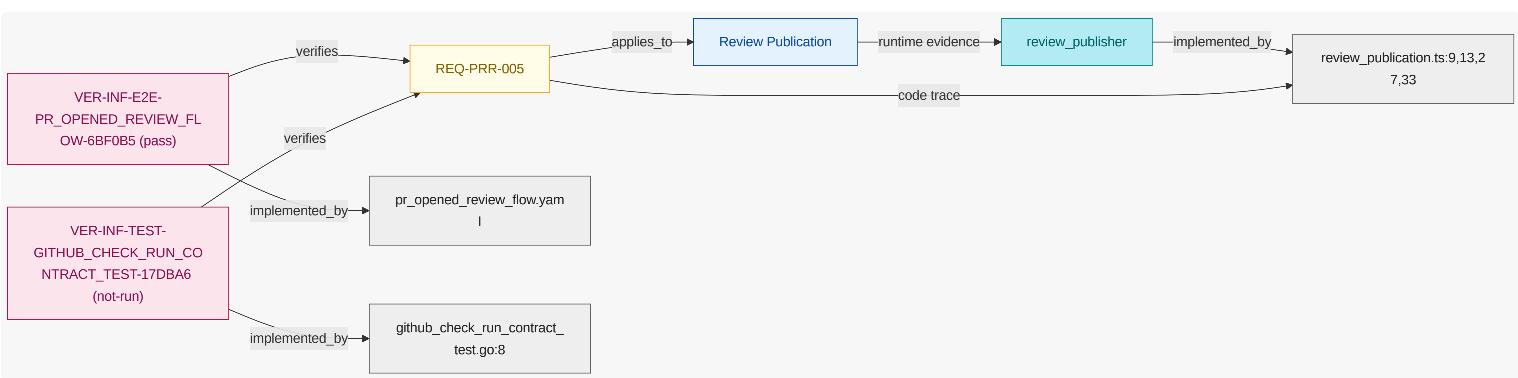

### 13.1.5. REQ-PRR-005

While `pull request review is active`, when `review findings are generated`, `the pr review system` shall publish `check run summary` to `repository maintainer`.

> **NOTE** Result `publication` contract.

#### Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



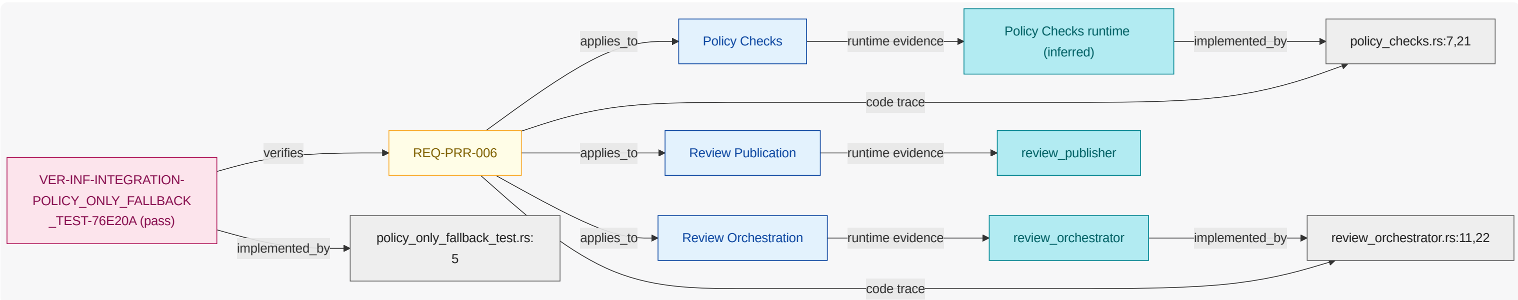

### 13.1.6. REQ-PRR-006

If `bedrock runtime is unavailable`, then `the pr review system` shall publish deterministic policy-only `review outcome`.

> **NOTE** Graceful degradation `requirement`.

#### Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



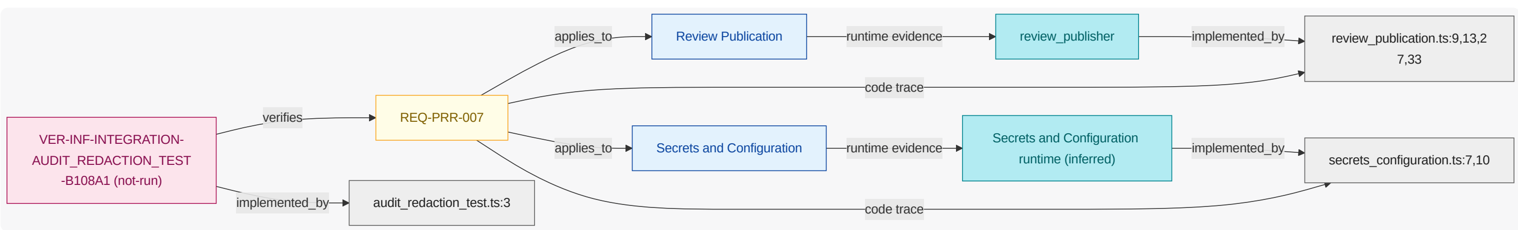

## 13.1.7. REQ-PRR-007

Where `audit logging is enabled`, when `review findings are published`, `the pr review system` shall persist `redacted audit metadata` for `security engineer` audit traceability.

> **NOTE** `Evidence` and audit lineage.

### Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



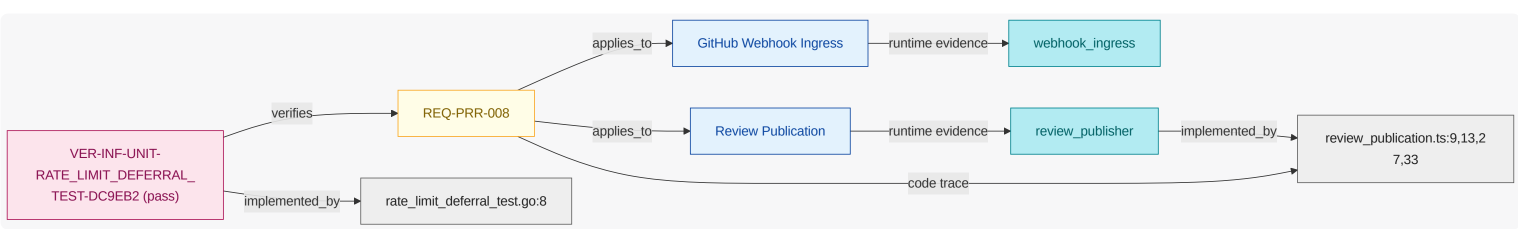

## 13.1.8. REQ-PRR-008

If `review request rate limit is exceeded`, then `the pr review system` shall defer processing.

> **NOTE** Abuse and capacity protection behavior.

### Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.




## 13.1.9. REQ-PRR-009

While `pull request review is active`, when `review findings are generated`, `the pr review system` shall publish inline comments with `result metadata` to `repository maintainer`.

> **NOTE** Inline-annotation `publication` contract.

### Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.


### 13.1.10. REQ-PRR-010

If `review request rate limit is exceeded`, then `the pr review system` shall notify `platform operator`.

> **NOTE** `Maintainer` notification for deferred review processing.

### Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.

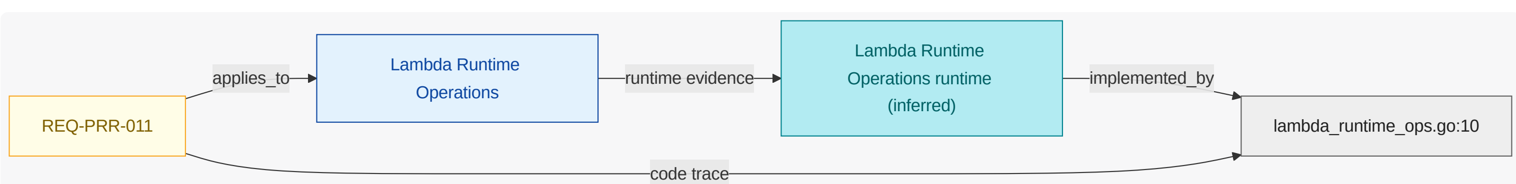

### 13.1.11. REQ-PRR-011

Where `lambda runtime operations are enabled`, when `release artifact is approved`, `the pr review system` shall deploy lambda function image to primary runtime.

> **NOTE** Runtime deployment lifecycle for the Lambda-hosted review application.

### Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.




## 13.2. Verification Inventory

What this section shows: verification checks inferred from test artifacts, linked to test code elements and mapped to verified requirements.

### 13.2.1. Pr Opened Review Flow

| Field | Value |
| --- | --- |
| ID | VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5 |
| Kind | e2e |
| Status | pass |
| Verifies Requirements | REQ-PRR-001, REQ-PRR-002, REQ-PRR-003, REQ-PRR-005 |
| Test Code Elements | tests/e2e/pr_opened_review_flow.yaml |
| Derived Functional Owners | FU-GITHUB-WEBHOOK-INGRESS, FU-PR-CONTEXT-ASSEMBLY, FU-REVIEW-ORCHESTRATION, FU-REVIEW-PUBLICATION |
| Result Summary | pass:4 |
| Evidence | test-results/contract-checkrun.json, test-results/e2e-pr-opened-review.xml, tests/e2e/pr_opened_review_flow.yaml |
| Notes | exercises end-to-end pull request opened flow from webhook ingest to published review |

#### 13.2.2. Audit Redaction Test

| Field | Value |
| --- | --- |
| ID | VER-INF-INTEGRATION-AUDIT_REDACTION_TEST-B108A1 |
| Kind | integration |
| Status | not-run |
| Verifies Requirements | REQ-PRR-007 |
| Test Code Elements | audit_redaction_test.ts:3 |
| Derived Functional Owners | FU-REVIEW-PUBLICATION, FU-SECRETS-CONFIGURATION |
| Result Summary | not-run:1 |
| Evidence | tests/integration/audit_redaction_test.ts |
| Notes | checks audit evidence redaction for secrets and sensitive payload fields |

#### 13.2.3. Policy Only Fallback Test

| Field | Value |
| --- | --- |
| ID | VER-INF-INTEGRATION-POLICY_ONLY_FALLBACK_TEST-76E20A |
| Kind | integration |
| Status | pass |
| Verifies Requirements | REQ-PRR-004, REQ-PRR-006 |

| Field | Value |
| --- | --- |
| Test Code Elements | policy_only_fallback_test.rs:5 |
| Derived Functional Owners | FU-POLICY-CHECKS, FU-REVIEW-ORCHESTRATION, FU-REVIEW-PUBLICATION |
| Result Summary | pass:2 |
| Evidence | test-results/integration-policy-fallback.json, tests/integration/policy_only_fallback_test.rs |
| Notes | verifies policy-only fallback when Bedrock inference is unavailable |

#### 13.2.4. Github Check Run Contract Test

| Field | Value |
| --- | --- |
| ID | VER-INF-TEST-GITHUB_CHECK_RUN_CONTRACT_TEST-17DBA6 |
| Kind | test |
| Status | not-run |
| Verifies Requirements | REQ-PRR-005 |
| Test Code Elements | github_check_run_contract_test.go:8 |
| Derived Functional Owners | FU-REVIEW-PUBLICATION |
| Result Summary | not-run:1 |
| Evidence | tests/contract/github_check_run_contract_test.go |
| Notes | validates GitHub check-run contract shape and required review result fields |

#### 13.2.5. Rate Limit Deferral Test

| Field | Value |
| --- | --- |
| ID | VER-INF-UNIT-RATE_LIMIT_DEFERRAL_TEST-DC9EB2 |
| Kind | unit |
| Status | pass |
| Verifies Requirements | REQ-PRR-008 |
| Test Code Elements | rate_limit_deferral_test.go:8 |
| Derived Functional Owners | FU-GITHUB-WEBHOOK-INGRESS, FU-REVIEW-PUBLICATION |
| Result Summary | pass:1 |
| Evidence | test-results/unit-rate-limit.xml, tests/unit/rate_limit_deferral_test.go |

| Field | Value |
| --- | --- |
| Notes | verifies rate-limit deferral behavior for pull request review processing |

# 14. Reference Index

## 14.1. Authored References

### CTRL-BEDROCK-SECRETS-ROTATION

category=credential-management; description=Rotate app and model credentials according to policy.

| Key | Value |
| --- | --- |
| Kind | Control |
| Name | Secret Rotation Enforcement |
| Mentioned In | Security View |

### CTRL-BEDROCK-WEBHOOK-SIGNATURE

category=input-integrity; description=Enforce signature verification for all GitHub webhook events.

| Key | Value |
| --- | --- |
| Kind | Control |
| Name | Webhook Signature Verification |
| Mentioned In | Security View |

### DO-BEDROCK-PR-CONTEXT

termRef=DATA-PR-CONTEXT; schemaRef=schemas/pr-context.json; sensitivity=internal

| Key | Value |
| --- | --- |
| Kind | Data Object |
| Name | Pull Request Context |
| Mentioned In | Architecture Intent View , Security View |

### DO-BEDROCK-REVIEW-FINDINGS

termRef=DATA-REVIEW-FINDINGS; schemaRef=schemas/review-findings.json; sensitivity=internal

| Key | Value |
| --- | --- |
| Kind | Data Object |
| Name | Review Findings |
| Mentioned In | Security View , REQ-PRR-003 |

### DEP-BEDROCK-GITHUB-BOUNDARY

environment=external; region=global; account=github; cluster=external; namespace=app; trustZone=external

| Key | Value |
| --- | --- |
| Kind | Deployment Target |
| Name | GitHub Integration Boundary |
| Mentioned In | Security View |

DEP-BEDROCK-LAMBDA-PRIMARY

environment=prod; region=us-east-1; account=app; cluster=lambda; namespace=review; trustZone=app

| Key | Value |
| --- | --- |
| Kind | Deployment Target |
| Name | Primary Lambda Runtime |
| Mentioned In | n/a |

EVT-BEDROCK-REVIEW-COMPLETED

Trigger indicating publication complete.

| Key | Value |
| --- | --- |
| Kind | Event |
| Name | Review Completed |
| Mentioned In | n/a |

EVT-BEDROCK-REVIEW-REQUESTED

Trigger for analysis orchestration.

| Key | Value |
| --- | --- |
| Kind | Event |
| Name | Review Requested |
| Mentioned In | n/a |

FLOW-BEDROCK-POLICY-TUNING

entry=update-policy-rules; exits=publish-policy-validation; steps=4

| Key | Value |
| --- | --- |
| Kind | Flow |
| Name | Security Policy Tuning Flow |
| Mentioned In | Security View |

### FLOW-BEDROCK-PR-REVIEW

entry=receive-webhook; exits=publish-review; steps=4

| Key | Value |
|---|---|
| Kind | Flow |
| Name | PR Review Interaction Flow |
| Mentioned In | Security View |

### FLOW-BEDROCK-POLICY-TUNING::orchestrate-policy-validation

kind=system_action; ref=FU-REVIEW-ORCHESTRATION; action=Orchestrate dry-run review validation with policy outputs; dataIn=policy_check_results; dataOut=validation_findings; async=false

| Key | Value |
|---|---|
| Kind | Flow Step |
| Name | Orchestrate dry-run review validation with policy outputs |
| Mentioned In | n/a |

### FLOW-BEDROCK-POLICY-TUNING::publish-policy-validation

kind=system_action; ref=FU-REVIEW-PUBLICATION; action=Publish policy validation findings to repository maintainers; dataIn=validation_findings; dataOut=; async=false

| Key | Value |
|---|---|
| Kind | Flow Step |
| Name | Publish policy validation findings to repository maintainers |
| Mentioned In | n/a |

### FLOW-BEDROCK-POLICY-TUNING::run-policy-checks

kind=system_action; ref=FU-POLICY-CHECKS; action=Execute deterministic policy checks on candidate rules; dataIn=; dataOut=policy_check_results; async=false

| Key | Value |
|---|---|
| Kind | Flow Step |
| Name | Execute deterministic policy checks on candidate rules |
| Mentioned In | n/a |

### FLOW-BEDROCK-POLICY-TUNING::update-policy-rules

kind=user_action; ref=ACT-SECURITY-ENGINEER; action=Security engineer updates review policy rules; dataIn=; dataOut=; async=false

| Key | Value |
| --- | --- |
| Kind | Flow Step |
| Name | Security engineer updates review policy rules |
| Mentioned In | n/a |

### FLOW-BEDROCK-PR-REVIEW::assemble-context

kind=system_action; ref=FU-PR-CONTEXT-ASSEMBLY; action=Build bounded review context; dataIn=; dataOut=pr_context; async=false

| Key | Value |
| --- | --- |
| Kind | Flow Step |
| Name | Build bounded review context |
| Mentioned In | n/a |

### FLOW-BEDROCK-PR-REVIEW::orchestrate-review

kind=system_action; ref=FU-REVIEW-ORCHESTRATION; action=Run AI + policy checks and prepare findings; dataIn=pr_context; dataOut=review_findings; async=true

| Key | Value |
| --- | --- |
| Kind | Flow Step |
| Name | Run AI + policy checks and prepare findings |
| Mentioned In | n/a |

### FLOW-BEDROCK-PR-REVIEW::publish-review

kind=system_action; ref=FU-REVIEW-PUBLICATION; action=Publish findings to GitHub check run; dataIn=review_findings; dataOut=; async=false

| Key | Value |
| --- | --- |
| Kind | Flow Step |
| Name | Publish findings to GitHub check run |
| Mentioned In | n/a |

### FLOW-BEDROCK-PR-REVIEW::receive-webhook

kind=user_action; ref=ACT-DEVELOPER; action=Developer pushes PR updates that trigger webhook processing; dataIn=; dataOut=; async=false

| Key | Value |
| --- | --- |
| Kind | Flow Step |

| Key | Value |
|---|---|
| Name | Developer pushes PR updates that trigger webhook processing |
| Mentioned In | n/a |

### IF-BEDROCK-REVIEW-PUBLISH-API

protocol=https; endpoint=/github/check-runs; schemaRef=schemas/review-output.json; owner=FU-REVIEW-PUBLICATION

| Key | Value |
|---|---|
| Kind | Interface |
| Name | Review Publish API |
| Mentioned In | n/a |

### IF-BEDROCK-WEBHOOK-API

protocol=https; endpoint=/webhook/github; schemaRef=schemas/webhook-event.json; owner=FU-GITHUB-WEBHOOK-INGRESS

| Key | Value |
|---|---|
| Kind | Interface |
| Name | Webhook Intake API |
| Mentioned In | Security View |

### REQ-PRR-001

While normal mode is active, when pull request synchronize event is received and webhook signature is valid, the pr review system shall verify webhook signature before processing review workflow.

| Key | Value |
|---|---|
| Kind | Requirement |
| Name | REQ-PRR-001 |
| Mentioned In | Architecture Intent View , Deployment View , Interaction Flow View , Pr Opened Review Flow |

### REQ-PRR-002

While pull request review is active, when pull request opened event is received, the pr review system shall construct diff context from changed files for developer review execution.

| Key | Value |
|---|---|
| Kind | Requirement |
| Name | REQ-PRR-002 |
| Mentioned In | Architecture Intent View , Deployment View , Interaction Flow View , Pr Opened Review Flow |

### REQ-PRR-003

Where ai review is enabled, when diff context is prepared, the pr review system shall request bedrock runtime analysis for review findings.

| Key | Value |
| --- | --- |
| Kind | Requirement |
| Name | REQ-PRR-003 |
| Mentioned In | Architecture Intent View, Deployment View, Interaction Flow View, Pr Opened Review Flow |

### REQ-PRR-004

Where policy checks are enabled, when pull request review is active, the pr review system shall evaluate deterministic policy checks for secret exposure plus infrastructure risk plus dependency changes.

| Key | Value |
| --- | --- |
| Kind | Requirement |
| Name | REQ-PRR-004 |
| Mentioned In | Architecture Intent View, Deployment View, Interaction Flow View, Policy Only Fallback Test |

### REQ-PRR-005

While pull request review is active, when review findings are generated, the pr review system shall publish check run summary to repository maintainer.

| Key | Value |
| --- | --- |
| Kind | Requirement |
| Name | REQ-PRR-005 |
| Mentioned In | Architecture Intent View, Deployment View, Interaction Flow View, Pr Opened Review Flow, Github Check Run Contract Test |

### REQ-PRR-006

If bedrock runtime is unavailable, then the pr review system shall publish deterministic policy-only review outcome.

| Key | Value |
| --- | --- |
| Kind | Requirement |
| Name | REQ-PRR-006 |
| Mentioned In | Architecture Intent View, Deployment View, Interaction Flow View, Policy Only Fallback Test |

### REQ-PRR-007

Where audit logging is enabled, when review findings are published, the pr review system shall persist redacted audit metadata for security engineer audit traceability.

| Key | Value |
| --- | --- |
| Kind | Requirement |
| Name | REQ-PRR-007 |
| Mentioned In | Architecture Intent View , Deployment View , Interaction Flow View , Audit Redaction Test |

### REQ-PRR-008

If review request rate limit is exceeded, then the pr review system shall defer processing.

| Key | Value |
| --- | --- |
| Kind | Requirement |
| Name | REQ-PRR-008 |
| Mentioned In | Architecture Intent View , Deployment View , Interaction Flow View , Rate Limit Deferral Test |

### REQ-PRR-009

While pull request review is active, when review findings are generated, the pr review system shall publish inline comments with result metadata to repository maintainer.

| Key | Value |
| --- | --- |
| Kind | Requirement |
| Name | REQ-PRR-009 |
| Mentioned In | Architecture Intent View , Deployment View |

### REQ-PRR-010

If review request rate limit is exceeded, then the pr review system shall notify platform operator.

| Key | Value |
| --- | --- |
| Kind | Requirement |
| Name | REQ-PRR-010 |
| Mentioned In | Architecture Intent View , Deployment View |

### REQ-PRR-011

Where lambda runtime operations are enabled, when release artifact is approved, the pr review system shall deploy lambda function image to primary runtime.

| Key | Value |
| --- | --- |
| Kind | Requirement |
| Name | REQ-PRR-011 |

| Key | Value |
| --- | --- |
| Mentioned In | Architecture Intent View , Deployment View |

### STATE-BEDROCK-PR-RECEIVED

PR webhook accepted and queued for analysis.

| Key | Value |
| --- | --- |
| Kind | State |
| Name | PR Received |
| Mentioned In | n/a |

### STATE-BEDROCK-REVIEW-PUBLISHED

Findings published to GitHub check run output.

| Key | Value |
| --- | --- |
| Kind | State |
| Name | Review Published |
| Mentioned In | n/a |

### TB-BEDROCK-AWS-CONTROL

Boundary between app workloads and privileged platform controls.

| Key | Value |
| --- | --- |
| Kind | Trust Boundary |
| Name | AWS Control Boundary |
| Mentioned In | Security View |

### TB-BEDROCK-GITHUB-EXTERNAL

Boundary between external GitHub traffic and internal review processing.

| Key | Value |
| --- | --- |
| Kind | Trust Boundary |
| Name | GitHub External Boundary |
| Mentioned In | n/a |

## 14.2. Catalog References

### ACT-DEVELOPER

Engineer opening and updating pull requests.

| Key | Value |
| --- | --- |
| Kind | Catalog Actor |
| Name | developer |
| Mentioned In | REQ-PRR-002 |

### ACT-PLATFORM-OPERATOR

Owner of deployment/runtime operations and reliability controls.

| Key | Value |
| --- | --- |
| Kind | Catalog Actor |
| Name | platform operator |
| Mentioned In | REQ-PRR-010 |

### ACT-REPOSITORY-MAINTAINER

Owner of merge decisions and repository quality gates.

| Key | Value |
| --- | --- |
| Kind | Catalog Actor |
| Name | repository maintainer |
| Mentioned In | REQ-PRR-005 , REQ-PRR-009 , REQ-PRR-010 |

### ACT-SECURITY-ENGINEER

Owner of security policy checks and finding triage.

| Key | Value |
| --- | --- |
| Kind | Catalog Actor |
| Name | security engineer |
| Mentioned In | Security View , REQ-PRR-007 |

### ai review enabled

Feature flag enabling model-assisted review.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | ai review enabled |

| Key | Value |
| --- | --- |
| Mentioned In | n/a |

## audit enabled

Feature flag enabling audit metadata persistence.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | audit enabled |
| Mentioned In | n/a |

## authorized events

Event raised when new commits are pushed to an open pull request.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | authorized events |
| Mentioned In | n/a |

## aws sdk

SDK dependency used for AWS runtime integrations.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | aws sdk |
| Mentioned In | n/a |

## bedrock api

External Bedrock runtime endpoint used for model inference.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | bedrock api |
| Mentioned In | n/a |

## bedrock unavailable

Event raised when Bedrock inference path is degraded.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | bedrock unavailable |
| Mentioned In | n/a |

### check run summaries

Aggregated review result posted as a GitHub check run.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | check run summaries |
| Mentioned In | n/a |

### context assembly

Fetches changed files and prepares bounded review context payloads.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | context assembly |
| Mentioned In | Architecture Intent View , Security View |

### context prepared

Event indicating context assembly completed and is ready for analysis.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | context prepared |
| Mentioned In | n/a |

### deterministic checks

Evaluates deterministic repository policy checks.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | deterministic checks |
| Mentioned In | Architecture Intent View |

### findings generated

Event indicating analysis output is ready for publication.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | findings generated |
| Mentioned In | n/a |

### findings published

Event indicating review output has been published to GitHub checks/comments.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | findings published |
| Mentioned In | n/a |

### github api

External GitHub API used for pull request metadata and review publication.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | github api |
| Mentioned In | `REQ-PRR-002` |

### github integration

Functional group for webhook ingress and pull request context intake.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | github integration |
| Mentioned In | n/a |

### github sdk

SDK dependency used for GitHub App REST interactions.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |

| Key | Value |
| --- | --- |
| Name | github sdk |
| Mentioned In | n/a |

### lambda ops

Manages runtime deployment and operational controls.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | lambda ops |
| Mentioned In | n/a |

### lambda runtime ops enabled

Feature flag enabling Lambda runtime deployment lifecycle operations.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | lambda runtime ops enabled |
| Mentioned In | n/a |

### maintainer

Owner of merge decisions and repository quality gates.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | maintainer |
| Mentioned In | n/a |

### normal mode

Default operating mode using AI plus deterministic checks.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | normal mode |
| Mentioned In | n/a |

### normalized review context

Pull request file changes and metadata prepared for analysis.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | normalized review context |
| Mentioned In | n/a |

## orchestration

Coordinates Bedrock analysis with deterministic checks and fallback logic.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | orchestration |
| Mentioned In | Architecture Intent View , Deployment View |

## platform ops

Owner of deployment/runtime operations and reliability controls.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | platform ops |
| Mentioned In | n/a |

## policy bundle

Deterministic review policy bundle used during analysis.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | policy bundle |
| Mentioned In | n/a |

## policy checks enabled

Feature flag enabling deterministic policy checks.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | policy checks enabled |

| Key | Value |
| --- | --- |
| Mentioned In | n/a |

### pr author

Engineer opening and updating pull requests.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | pr author |
| Mentioned In | n/a |

### pr opened

Event raised when a new pull request is opened.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | pr opened |
| Mentioned In | n/a |

### pr spam

High-volume event abuse causing review capacity exhaustion.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | pr spam |
| Mentioned In | n/a |

### pr synchronize

Event raised when new commits are pushed to an open pull request.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | pr synchronize |
| Mentioned In | n/a |

### prompt injection

Malicious prompt-like content embedded in pull request code changes.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | prompt injection |
| Mentioned In | Security View |

### publication

Publishes review findings to GitHub checks and comments.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | publication |
| Mentioned In | Architecture Intent View , Deployment View , Security View , REQ-PRR-005 , REQ-PRR-009 |

### publication output

Event indicating review output has been published to GitHub checks/comments.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | publication output |
| Mentioned In | n/a |

### pull request files

Pull request file changes and metadata prepared for analysis.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | pull request files |
| Mentioned In | n/a |

### pull request metadata

Pull request file changes and metadata prepared for analysis.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | pull request metadata |
| Mentioned In | n/a |

### rate limit exceeded

Event raised when PR review rate guardrails are hit.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | rate limit exceeded |
| Mentioned In | n/a |

### release approved

Event raised when a Lambda runtime release artifact has passed approval.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | release approved |
| Mentioned In | n/a |

### review active

State where review processing is currently active for a pull request.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | review active |
| Mentioned In | n/a |

### review context

Pull request file changes and metadata prepared for analysis.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | review context |
| Mentioned In | Architecture Intent View |

### review outcome

Event indicating analysis output is ready for publication.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |

| Key | Value |
| --- | --- |
| Name | review outcome |
| Mentioned In | REQ-PRR-006 |

### review system

Functional group for AI and deterministic review analysis and publication.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | review system |
| Mentioned In | n/a |

### review-ready context

Pull request file changes and metadata prepared for analysis.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | review-ready context |
| Mentioned In | n/a |

### runtime operations

Functional group for deployment lifecycle and secret/config controls.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | runtime operations |
| Mentioned In | Architecture Intent View |

### secret config

Manages credentials and secure runtime configuration.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | secret config |
| Mentioned In | n/a |

### secret leakage

Unintended credential disclosure in generated review findings.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | secret leakage |
| Mentioned In | n/a |

### secrets manager

Managed secret store for application credentials and config values.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | secrets manager |
| Mentioned In | n/a |

### security reviewer

Owner of security policy checks and finding triage.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | security reviewer |
| Mentioned In | n/a |

### signature valid

Condition indicating webhook HMAC verification succeeded.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | signature valid |
| Mentioned In | n/a |

### the pr review system

GitHub App review system-of-interest that ingests PR events and publishes review outcomes.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | the pr review system |

| Key | Value |
| --- | --- |
| Mentioned In | REQ-PRR-001, REQ-PRR-002, REQ-PRR-003, REQ-PRR-004, REQ-PRR-005, REQ-PRR-006, REQ-PRR-007, REQ-PRR-008, REQ-PRR-009, REQ-PRR-010, REQ-PRR-011 |

### webhook events

Event raised when a new pull request is opened.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | webhook events |
| Mentioned In | n/a |

### webhook ingress

Validates webhook signatures and routes trusted pull request events.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | webhook ingress |
| Mentioned In | Deployment View |

### webhook spoofing

Forged webhook request attempting unauthorized review execution.

| Key | Value |
| --- | --- |
| Kind | Catalog Alias |
| Name | webhook spoofing |
| Mentioned In | n/a |

### AV-PR-SPAM-ABUSE

High-volume event abuse causing review capacity exhaustion.

| Key | Value |
| --- | --- |
| Kind | Catalog Attack Vector |
| Name | pull request spam abuse |
| Mentioned In | n/a |

### AV-PROMPT-INJECTION-IN-DIFF

Malicious prompt-like content embedded in pull request code changes.

| Key | Value |
|---|---|
| Kind | Catalog Attack Vector |
| Name | prompt injection in diff |
| Mentioned In | Security View |

### AV-SECRET-LEAKAGE-IN-REVIEW

Unintended credential disclosure in generated review findings.

| Key | Value |
|---|---|
| Kind | Catalog Attack Vector |
| Name | secret leakage in review output |
| Mentioned In | Security View |

### AV-SPOOFED-GITHUB-WEBHOOK

Forged webhook request attempting unauthorized review execution.

| Key | Value |
|---|---|
| Kind | Catalog Attack Vector |
| Name | spoofed github webhook |
| Mentioned In | Security View |

### COND-WEBHOOK-SIGNATURE-VALID

Condition indicating webhook HMAC verification succeeded.

| Key | Value |
|---|---|
| Kind | Catalog Condition |
| Name | webhook signature is valid |
| Mentioned In | REQ-PRR-001 |

### DATA-CHECKRUN-SUMMARY

Aggregated review result posted as a GitHub check run.

| Key | Value |
|---|---|
| Kind | Catalog Data Term |
| Name | check run summary |
| Mentioned In | REQ-PRR-005 |

### DATA-DIFF-CONTEXT

Pull request file changes and metadata prepared for analysis.

| Key | Value |
| --- | --- |
| Kind | Catalog Data Term |
| Name | diff context |
| Mentioned In | REQ-PRR-002 |

### DATA-REDACTED-AUDIT-METADATA

Sanitized prompt and finding metadata retained for audit.

| Key | Value |
| --- | --- |
| Kind | Catalog Data Term |
| Name | redacted audit metadata |
| Mentioned In | REQ-PRR-007 |

### DATA-RESULT-METADATA

Metadata describing review output characteristics for audit and reporting.

| Key | Value |
| --- | --- |
| Kind | Catalog Data Term |
| Name | result metadata |
| Mentioned In | REQ-PRR-009 |

### DATA-WEBHOOK-SIGNATURE

Signature used to verify webhook authenticity.

| Key | Value |
| --- | --- |
| Kind | Catalog Data Term |
| Name | webhook signature |
| Mentioned In | Architecture Intent View , Security View , REQ-PRR-001 |

### EVT-BEDROCK-UNAVAILABLE

Event raised when Bedrock inference path is degraded.

| Key | Value |
| --- | --- |
| Kind | Catalog Event |

| Key | Value |
|---|---|
| Name | bedrock runtime is unavailable |
| Mentioned In | REQ-PRR-006 |

### EVT-CONTEXT-PREPARED

Event indicating context assembly completed and is ready for analysis.

| Key | Value |
|---|---|
| Kind | Catalog Event |
| Name | diff context is prepared |
| Mentioned In | REQ-PRR-003 |

### EVT-FINDINGS-GENERATED

Event indicating analysis output is ready for publication.

| Key | Value |
|---|---|
| Kind | Catalog Event |
| Name | review findings are generated |
| Mentioned In | REQ-PRR-005 , REQ-PRR-009 |

### EVT-FINDINGS-PUBLISHED

Event indicating review output has been published to GitHub checks/comments.

| Key | Value |
|---|---|
| Kind | Catalog Event |
| Name | review findings are published |
| Mentioned In | REQ-PRR-007 |

### EVT-PR-OPENED

Event raised when a new pull request is opened.

| Key | Value |
|---|---|
| Kind | Catalog Event |
| Name | pull request opened event is received |
| Mentioned In | REQ-PRR-002 |

### EVT-PR-SYNCHRONIZE

Event raised when new commits are pushed to an open pull request.

| Key | Value |
|---|---|
| Kind | Catalog Event |
| Name | pull request synchronize event is received |
| Mentioned In | `REQ-PRR-001` |

### EVT-RATE-LIMIT-EXCEEDED

Event raised when PR review rate guardrails are hit.

| Key | Value |
|---|---|
| Kind | Catalog Event |
| Name | review request rate limit is exceeded |
| Mentioned In | `REQ-PRR-008`, `REQ-PRR-010` |

### EVT-RELEASE-APPROVED

Event raised when a Lambda runtime release artifact has passed approval.

| Key | Value |
|---|---|
| Kind | Catalog Event |
| Name | release artifact is approved |
| Mentioned In | `REQ-PRR-011` |

### FEAT-AI-REVIEW

Feature flag enabling model-assisted review.

| Key | Value |
|---|---|
| Kind | Catalog Feature |
| Name | ai review is enabled |
| Mentioned In | `REQ-PRR-003` |

### FEAT-AUDIT-LOGGING

Feature flag enabling audit metadata persistence.

| Key | Value |
|---|---|
| Kind | Catalog Feature |
| Name | audit logging is enabled |

| Key | Value |
| --- | --- |
| Mentioned In | REQ-PRR-007 |

### FEAT-LAMBDA-RUNTIME-OPS

Feature flag enabling Lambda runtime deployment lifecycle operations.

| Key | Value |
| --- | --- |
| Kind | Catalog Feature |
| Name | lambda runtime operations are enabled |
| Mentioned In | REQ-PRR-011 |

### FEAT-POLICY-CHECKS

Feature flag enabling deterministic policy checks.

| Key | Value |
| --- | --- |
| Kind | Catalog Feature |
| Name | policy checks are enabled |
| Mentioned In | REQ-PRR-004 |

### FG-PLATFORM-OPERATIONS

Functional group for deployment lifecycle and secret/config controls.

| Key | Value |
| --- | --- |
| Kind | Catalog Functional Group |
| Name | platform operations |
| Mentioned In | Architecture Intent View , Deployment View |

### FG-PR-INTEGRATION

Functional group for webhook ingress and pull request context intake.

| Key | Value |
| --- | --- |
| Kind | Catalog Functional Group |
| Name | pr integration |
| Mentioned In | Architecture Intent View , Deployment View |

### FG-REVIEW-INTELLIGENCE

Functional group for AI and deterministic review analysis and publication.

| Key | Value |
| --- | --- |
| Kind | Catalog Functional Group |
| Name | review intelligence |
| Mentioned In | Architecture Intent View , Deployment View |

### FU-GITHUB-WEBHOOK-INGRESS

Validates webhook signatures and routes trusted pull request events.

| Key | Value |
| --- | --- |
| Kind | Catalog Functional Unit |
| Name | github webhook ingress |
| Mentioned In | Architecture Intent View , Security View , Pr Opened Review Flow , Rate Limit Deferral Test |

### FU-LAMBDA-RUNTIME-OPERATIONS

Manages runtime deployment and operational controls.

| Key | Value |
| --- | --- |
| Kind | Catalog Functional Unit |
| Name | lambda runtime operations |
| Mentioned In | Architecture Intent View , Communication View , Deployment View , Security View , Traceability View |

### FU-POLICY-CHECKS

Evaluates deterministic repository policy checks.

| Key | Value |
| --- | --- |
| Kind | Catalog Functional Unit |
| Name | policy checks |
| Mentioned In | Architecture Intent View , Communication View , Deployment View , Security View , Traceability View , Interaction Flow View , REQ-PRR-004 , Policy Only Fallback Test |

### FU-PR-CONTEXT-ASSEMBLY

Fetches changed files and prepares bounded review context payloads.

| Key | Value |
| --- | --- |
| Kind | Catalog Functional Unit |
| Name | pr context assembly |

| Key | Value |
| --- | --- |
| Mentioned In | Architecture Intent View, Communication View, Deployment View, Security View, Traceability View, Pr Opened Review Flow |

### FU-REVIEW-ORCHESTRATION

Coordinates Bedrock analysis with deterministic checks and fallback logic.

| Key | Value |
| --- | --- |
| Kind | Catalog Functional Unit |
| Name | review orchestration |
| Mentioned In | Architecture Intent View, Deployment View, Security View, Pr Opened Review Flow, Policy Only Fallback Test |

### FU-REVIEW-PUBLICATION

Publishes review findings to GitHub checks and comments.

| Key | Value |
| --- | --- |
| Kind | Catalog Functional Unit |
| Name | review publication |
| Mentioned In | Architecture Intent View, Deployment View, Security View, Pr Opened Review Flow, Audit Redaction Test, Policy Only Fallback Test, Github Check Run Contract Test, Rate Limit Deferral Test |

### FU-SECRETS-CONFIGURATION

Manages credentials and secure runtime configuration.

| Key | Value |
| --- | --- |
| Kind | Catalog Functional Unit |
| Name | secrets and configuration |
| Mentioned In | Architecture Intent View, Communication View, Deployment View, Security View, Traceability View, Audit Redaction Test |

### MODE-NORMAL

Default operating mode using AI plus deterministic checks.

| Key | Value |
| --- | --- |
| Kind | Catalog Mode |
| Name | normal mode is active |
| Mentioned In | REQ-PRR-001 |

### REF-AWS-BEDROCK-RUNTIME

External Bedrock runtime endpoint used for model inference.

| Key | Value |
| --- | --- |
| Kind | Catalog Referenced Element |
| Name | bedrock runtime |
| Mentioned In | Architecture Intent View , Deployment View , Security View , REQ-PRR-003 |

### REF-AWS-SDK-GO

SDK dependency used for AWS runtime integrations.

| Key | Value |
| --- | --- |
| Kind | Catalog Referenced Element |
| Name | aws sdk for go |
| Mentioned In | Architecture Intent View , Deployment View , Security View |

### REF-AWS-SECRETS-MANAGER

Managed secret store for application credentials and config values.

| Key | Value |
| --- | --- |
| Kind | Catalog Referenced Element |
| Name | aws secrets manager |
| Mentioned In | Architecture Intent View , Deployment View , Security View |

### REF-GITHUB-APP-API

External GitHub API used for pull request metadata and review publication.

| Key | Value |
| --- | --- |
| Kind | Catalog Referenced Element |
| Name | github app api |
| Mentioned In | Architecture Intent View , Deployment View , Security View |

### REF-GITHUB-REST-SDK

SDK dependency used for GitHub App REST interactions.

| Key | Value |
| --- | --- |
| Kind | Catalog Referenced Element |

| Key | Value |
| --- | --- |
| Name | github rest sdk |
| Mentioned In | Architecture Intent View , Deployment View , Security View |

### REF-POLICY-RULESET

Deterministic review policy bundle used during analysis.

| Key | Value |
| --- | --- |
| Kind | Catalog Referenced Element |
| Name | policy ruleset |
| Mentioned In | Architecture Intent View , Deployment View |

### STATE-PR-REVIEW-ACTIVE

State where review processing is currently active for a pull request.

| Key | Value |
| --- | --- |
| Kind | Catalog State |
| Name | pull request review is active |
| Mentioned In | REQ-PRR-002 , REQ-PRR-004 , REQ-PRR-005 , REQ-PRR-009 |

### SYS-PR-REVIEW-SYSTEM

GitHub App review system-of-interest that ingests PR events and publishes review outcomes.

| Key | Value |
| --- | --- |
| Kind | Catalog System |
| Name | pr review system |
| Mentioned In | n/a |

### TERM-ACTOR

A person or operational role that interacts with functional units.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | actor |
| Mentioned In | n/a |

### TERM-ASCIIDOC-DIAGRAM

Generated diagram block, usually Mermaid, that visualizes architecture relationships in the AsciiDoc publication.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | AsciiDoc diagram |
| Mentioned In | n/a |

### TERM-ASCIIDOC-DOCUMENT

Human-readable generated architecture publication document assembled from authored model, inferred evidence, diagrams, references, and decisions.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | AsciiDoc document |
| Mentioned In | n/a |

### TERM-ASCIIDOC-SECTION

Structured generated document section or row model used to render authored and inferred architecture content.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | AsciiDoc section |
| Mentioned In | n/a |

### TERM-ATTACK-VECTOR

A technical misuse or attack path that targets functional, referenced, or runtime elements.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | attack vector |
| Mentioned In | n/a |

### TERM-AUTHORED-MAPPING

An explicit relationship declared in architecture inputs between authored or referenced elements.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |

| Key | Value |
|---|---|
| Name | authored mapping |
| Mentioned In | `Communication View`, `Security View` |

TERM-BUNDLE

Loaded architecture, catalog, and companion documents resolved as one working model context.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | model bundle |
| Mentioned In | n/a |

TERM-CATALOG

Controlled vocabulary document used to normalize model, requirement, and generated-document terminology.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | catalog |
| Mentioned In | n/a |

TERM-CATALOG-ENTRY

A stable catalog term with a definition and aliases for linting, linking, and generated references.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | catalog entry |
| Mentioned In | n/a |

TERM-CLI-COMMAND

Command-line entrypoint that coordinates loading, validation, generation, and file output for an engineering-model workflow.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | CLI command |
| Mentioned In | n/a |

TERM-CODE-ELEMENT

An inferred code structure or ownership element discovered from source trees and build metadata.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | code element |
| Mentioned In | n/a |

### TERM-COMPLIANCE-MAPPING

Authored mapping that links a model control to selected compliance controls, model scope, implementation status, evidence, and responsible roles.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | compliance mapping |
| Mentioned In | n/a |

### TERM-CONTROL

An authored security, policy, or operational control used to constrain behavior and reduce risk.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | control |
| Mentioned In | `Deployment View` , `Security View` |

### TERM-CONTROL-VERIFICATION

Authored control verification record with method, status, threat/risk scope, findings, and evidence.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | control verification |
| Mentioned In | n/a |

### TERM-DATA-OBJECT

An authored data artifact or contract shape traced across flow, interface, and implementation boundaries.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |

| Key | Value |
|---|---|
| Name | data object |
| Mentioned In | n/a |

### TERM-DECISION

Authored architecture decision record with status, context, decision text, and consequences.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | architecture decision |
| Mentioned In | n/a |

### TERM-DEPLOYMENT-TARGET

An authored deployment destination such as environment, cluster, namespace, or registry scope.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | deployment target |
| Mentioned In | n/a |

### TERM-DESIGN

Authored design narrative organized by model entities and architecture views.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | design document |
| Mentioned In | n/a |

### TERM-DESIGN-VIEW

View-scoped design narrative attached to an authored functional group or functional unit.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | design view |
| Mentioned In | n/a |

### TERM-EVENT

An authored trigger signal that drives transitions, flow progress, or asynchronous outcomes.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | event |
| Mentioned In | Architecture Intent View, Communication View, Security View, State Lifecycle View |

### TERM-EVIDENCE

A file, artifact, or observation used to support control, risk, threat, or verification claims.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | evidence |
| Mentioned In | Communication View, Deployment View, Security View, Traceability View, State Lifecycle View, Interaction Flow View, REQ-PRR-007, Audit Redaction Test |

### TERM-FLOW

An authored causal interaction sequence from user/system intent to outcome, represented as ordered steps.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | flow |
| Mentioned In | Architecture Intent View, Communication View, Deployment View, Interaction Flow View, Pr Opened Review Flow |

### TERM-FLOW-STEP

A single authored step in a flow that captures action, data in/out, references, and normal/error/async transitions.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | flow step |
| Mentioned In | n/a |

### TERM-FUNCTIONAL-GROUP

A major authored capability area that groups related functional units.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |

| Key | Value |
| --- | --- |
| Name | functional group |
| Mentioned In | n/a |

### TERM-FUNCTIONAL-UNIT

An authored working unit inside a functional group that owns specific behavior.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | functional unit |
| Mentioned In | State Lifecycle View |

### TERM-GENERATION-WORKFLOW

Orchestrated run that combines model loading, validation, projection, rendering, and artifact emission.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | generation workflow |
| Mentioned In | n/a |

### TERM-INFERENCE-HINT

Authored source and ownership configuration used to discover runtime, code, and verification evidence.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | inference hint |
| Mentioned In | n/a |

### TERM-INFERRED-MAPPING

A discovered relationship that links inferred runtime/code elements upward to authored design.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | inferred mapping |
| Mentioned In | n/a |

### TERM-INTERFACE

An authored interface boundary (for example API, channel, endpoint, or contract) owned by a functional unit.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | interface |
| Mentioned In | Communication View |

### TERM-LINT-RUN

Requirement lint configuration that controls parsing and quality checks for requirement text.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | lint run |
| Mentioned In | n/a |

### TERM-LOBSTER-ACTIVITY-TRACE

Generated LOBSTER activity JSON that links verification evidence back to formal requirement identifiers.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | LOBSTER activity trace |
| Mentioned In | n/a |

### TERM-MCP-SERVER

Model Context Protocol server that exposes engineering-model operations to AI agents through JSON-RPC tools.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | MCP server |
| Mentioned In | n/a |

### TERM-MCP-STDIO-TRANSPORT

Content-Length framed standard input/output transport used by the MCP command-line server.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | MCP stdio transport |

| Key | Value |
| --- | --- |
| Mentioned In | n/a |

### TERM-MCP-TOOL

Named MCP operation with a constrained input schema and structured response payload.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | MCP tool |
| Mentioned In | n/a |

### TERM-MCP-TOOL-RESPONSE

Structured MCP tool payload that includes success state, schema version, generated timestamp, and result data or validation error.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | MCP tool response |
| Mentioned In | n/a |

### TERM-MODEL

The authored architecture model root that composes functional structure, relationships, inference hints, and views.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | model |
| Mentioned In | `Deployment View`, `Security View` |

### TERM-OPEN-OTM-DOCUMENT

Open Threat Model JSON representation generated from the engineering model for interoperable threat-model exchange.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | Open OTM document |
| Mentioned In | n/a |

### TERM-OSCAL-ASSESSMENT-RESULTS

Generated OSCAL assessment results that summarize reviewed controls, verification findings, and modeled risks.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | OSCAL assessment results |
| Mentioned In | n/a |

### TERM-OSCAL-POAM

Generated OSCAL plan of action and milestones that maps modeled POA&M items and related risks into compliance JSON.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | OSCAL POA&M |
| Mentioned In | n/a |

### TERM-OSCAL-SSP

Generated OSCAL system security plan that maps authored system metadata, components, controls, allocations, and evidence into SSP JSON.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | OSCAL SSP |
| Mentioned In | n/a |

### TERM-POAM-ITEM

Plan of action and milestones item linked to a modeled risk and supporting artifacts.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | POA&M item |
| Mentioned In | n/a |

### TERM-REFERENCE-INDEX

Generated index of authored, catalog, runtime, code, and verification references with stable anchors and backlinks.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | reference index |

| Key | Value |
|---|---|
| Mentioned In | n/a |

### TERM-REFERENCED-ELEMENT

An architecture-relevant external, platform, or third-party dependency represented by role.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | referenced element |
| Mentioned In | n/a |

### TERM-REPO-INDEX

Bounded first-party source index used to answer model-aware file, requirement, control, and threat lookup queries.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | repository index |
| Mentioned In | n/a |

### TERM-REQUIREMENT

A structured requirement statement that defines expected system behavior and maps to functional ownership.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | requirement |
| Mentioned In | Traceability View , Interaction Flow View , REQ-PRR-006 |

### TERM-RISK

Authored risk record with likelihood, impact, response, scope, controls, and evidence.

| Key | Value |
|---|---|
| Kind | Engineering Model Term |
| Name | risk |
| Mentioned In | Security View , REQ-PRR-004 |

### TERM-RUNTIME-ELEMENT

An inferred runtime realization element discovered from infrastructure and deployment sources.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | runtime element |
| Mentioned In | n/a |

### TERM-SECURITY-CONTEXT

Security-focused generated context view that groups owned functional units and shows external actors, references, flows, controls, and trust boundaries.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | security context |
| Mentioned In | n/a |

### TERM-SOURCE-BLOCK

Stable source reference block that records file path, optional line span, kind, summary, and linked entity IDs.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | source block |
| Mentioned In | n/a |

### TERM-STATE

An authored lifecycle state used to model transition behavior, guards, and event-driven progression.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | state |
| Mentioned In | State Lifecycle View |

### TERM-STRUCTURIZR-DSL

Generated Structurizr DSL workspace that projects authored architecture, relationships, dynamic views, and deployment metadata.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | Structurizr DSL |

| Key | Value |
| --- | --- |
| Mentioned In | n/a |

### TERM-THREAT-ASSUMPTION

Authored security assumption that records scope, rationale, owner, status, and evidence.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | threat assumption |
| Mentioned In | n/a |

### TERM-THREAT-DRAGON-DOCUMENT

Threat Dragon JSON representation generated from the engineering model for STRIDE threat-model review.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | Threat Dragon document |
| Mentioned In | n/a |

### TERM-THREAT-MITIGATION

Authored mitigation record that links a threat scenario to a control, effectiveness, verification, and evidence.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | threat mitigation |
| Mentioned In | n/a |

### TERM-THREAT-MODEL-EXPORT

Generated security artifact that translates authored architecture, flows, trust boundaries, threats, and mitigations into an external threat-model schema.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | threat model export |
| Mentioned In | n/a |

### TERM-THREAT-OUT-OF-SCOPE

Authored decision that excludes a threat concern from current scope with reason, owner, expiry, and evidence.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | threat out-of-scope decision |
| Mentioned In | n/a |

### TERM-THREAT-SCENARIO

Authored threat narrative that connects attack vectors, scope, flows, controls, risks, and verification evidence.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | threat scenario |
| Mentioned In | n/a |

### TERM-TRACE-MARKER

Source-level marker such as TRLC-LINKS or ENGMODEL-LINKS used to connect declarations to requirements and model entities.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | trace marker |
| Mentioned In | n/a |

### TERM-TRLC-PACKAGE

Generated TRLC model and requirements package used for formal requirement traceability processing.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | TRLC package |
| Mentioned In | n/a |

### TERM-TRUST-BOUNDARY

An authored trust separation zone that marks policy, identity, or security control boundaries.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | trust boundary |
| Mentioned In | `Architecture Intent View`, `Security View`, `REQ-PRR-001` |

### TERM-UPWARD-LINKING

Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | upward linking |
| Mentioned In | n/a |

### TERM-VALIDATION

Model quality gate that checks authored documents, references, relationship semantics, and requirement lint results.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | validation |
| Mentioned In | `REQ-PRR-001` |

### TERM-VALIDATION-DIAGNOSTIC

Structured validation finding with code, severity, message, and optional source path.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | validation diagnostic |
| Mentioned In | n/a |

### TERM-VERIFICATION-CHECK

An inferred or authored verification artifact that validates requirement behavior with test evidence.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |
| Name | verification check |
| Mentioned In | n/a |

### TERM-VIEW

Authored projection configuration that selects roots, entity kinds, mappings, audience, and abstraction.

| Key | Value |
| --- | --- |
| Kind | Engineering Model Term |

| Key | Value |
|---|---|
| Name | view |
| Mentioned In | Communication View, Deployment View, Security View, Traceability View, State Lifecycle View, Interaction Flow View |

## 14.3. Inferred Runtime References

### review_orchestrator

assembles diff context and orchestrates deterministic plus Bedrock-backed review analysis

| Key | Value |
|---|---|
| Kind | Runtime lambda_function |
| Owner | FU-REVIEW-ORCHESTRATION |
| Source | examples/bedrock-pr-review-github-app-sample/infra/terraform/main.tf |
| Mentioned In | Deployment View, Security View |

### review_publisher

publishes check run summaries and inline review comments back to GitHub

| Key | Value |
|---|---|
| Kind | Runtime lambda_function |
| Owner | FU-REVIEW-PUBLICATION |
| Source | examples/bedrock-pr-review-github-app-sample/infra/terraform/main.tf |
| Mentioned In | Deployment View, Security View |

### webhook_ingress

validates GitHub webhook signature and normalizes pull request event payloads

| Key | Value |
|---|---|
| Kind | Runtime lambda_function |
| Owner | FU-GITHUB-WEBHOOK-INGRESS |
| Source | examples/bedrock-pr-review-github-app-sample/infra/terraform/main.tf |
| Mentioned In | Deployment View, Security View |

## 14.4. Inferred Code References

### aws-sdk-go-v2/secretsmanager (secrets_configuration.ts)

Inferred library_external dependency owned by FU-SECRETS-CONFIGURATION from secrets_configuration.ts.

| Key | Value |
| --- | --- |
| Kind | Code library_external |
| Owner | FU-SECRETS-CONFIGURATION |
| Source | secrets_configuration.ts |
| Mentioned In | n/a |

### github-rest-sdk (pr_context_assembly.ts)

Inferred library_external dependency owned by FU-PR-CONTEXT-ASSEMBLY from pr_context_assembly.ts.

| Key | Value |
| --- | --- |
| Kind | Code library_external |
| Owner | FU-PR-CONTEXT-ASSEMBLY |
| Source | pr_context_assembly.ts |
| Mentioned In | n/a |

### github-rest-sdk/publication-client (review_publication.ts)

Inferred library_external dependency owned by FU-REVIEW-PUBLICATION from review_publication.ts.

| Key | Value |
| --- | --- |
| Kind | Code library_external |
| Owner | FU-REVIEW-PUBLICATION |
| Source | review_publication.ts |
| Mentioned In | n/a |

### serde_json::json (review_orchestrator.rs)

Inferred library_external dependency owned by FU-REVIEW-ORCHESTRATION from review_orchestrator.rs.

| Key | Value |
| --- | --- |
| Kind | Code library_external |
| Owner | FU-REVIEW-ORCHESTRATION |
| Source | review_orchestrator.rs |
| Mentioned In | n/a |

### crypto/hmac (webhook_ingress.go)

Inferred library_stdlib dependency owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go.

| Key | Value |
| --- | --- |
| Kind | Code library_stdlib |
| Owner | FU-GITHUB-WEBHOOK-INGRESS |
| Source | webhook_ingress.go |
| Mentioned In | n/a |

### crypto/sha256 (webhook_ingress.go)

Inferred library_stdlib dependency owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go.

| Key | Value |
| --- | --- |
| Kind | Code library_stdlib |
| Owner | FU-GITHUB-WEBHOOK-INGRESS |
| Source | webhook_ingress.go |
| Mentioned In | n/a |

### encoding/hex (webhook_ingress.go)

Inferred library_stdlib dependency owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go.

| Key | Value |
| --- | --- |
| Kind | Code library_stdlib |
| Owner | FU-GITHUB-WEBHOOK-INGRESS |
| Source | webhook_ingress.go |
| Mentioned In | n/a |

### fmt (webhook_ingress.go)

Inferred library_stdlib dependency owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go.

| Key | Value |
| --- | --- |
| Kind | Code library_stdlib |
| Owner | FU-GITHUB-WEBHOOK-INGRESS |
| Source | webhook_ingress.go |
| Mentioned In | n/a |

### log (lambda_runtime_ops.go)

Inferred library_stdlib dependency owned by FU-LAMBDA-RUNTIME-OPERATIONS from lambda_runtime_ops.go.

| Key | Value |
|---|---|
| Kind | Code library_stdlib |
| Owner | FU-LAMBDA-RUNTIME-OPERATIONS |
| Source | lambda_runtime_ops.go |
| Mentioned In | n/a |

### lambda_runtime_ops.go

wires Lambda runtime entrypoints and shared execution dependencies

| Key | Value |
|---|---|
| Kind | Code source_file |
| Owner | FU-LAMBDA-RUNTIME-OPERATIONS |
| Source | lambda_runtime_ops.go |
| Mentioned In | Deployment View |

### policy_checks.rs

evaluates policy and style rules before review publication

| Key | Value |
|---|---|
| Kind | Code source_file |
| Owner | FU-POLICY-CHECKS |
| Source | policy_checks.rs |
| Mentioned In | Deployment View |

### pr_context_assembly.ts

collects pull request diff context and prepares review input payloads

| Key | Value |
|---|---|
| Kind | Code source_file |
| Owner | FU-PR-CONTEXT-ASSEMBLY |
| Source | pr_context_assembly.ts |
| Mentioned In | Deployment View , Security View |

### review_orchestrator.rs

coordinates deterministic and AI review steps for pull requests

| Key | Value |
| --- | --- |
| Kind | Code source_file |
| Owner | FU-REVIEW-ORCHESTRATION |
| Source | review_orchestrator.rs |
| Mentioned In | n/a |

### review_publication.ts

publishes review comments and check-run outcomes back to GitHub

| Key | Value |
| --- | --- |
| Kind | Code source_file |
| Owner | FU-REVIEW-PUBLICATION |
| Source | review_publication.ts |
| Mentioned In | Deployment View , Security View |

### secrets_configuration.ts

loads GitHub and Bedrock secret configuration for review execution

| Key | Value |
| --- | --- |
| Kind | Code source_file |
| Owner | FU-SECRETS-CONFIGURATION |
| Source | secrets_configuration.ts |
| Mentioned In | Deployment View , Security View |

### webhook_ingress.go

validates GitHub webhook signatures and routes pull request events

| Key | Value |
| --- | --- |
| Kind | Code source_file |
| Owner | FU-GITHUB-WEBHOOK-INGRESS |
| Source | webhook_ingress.go |
| Mentioned In | n/a |

### lambda_runtime_ops.go:10

Inferred code symbol owned by FU-LAMBDA-RUNTIME-OPERATIONS from lambda_runtime_ops.go:10.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-LAMBDA-RUNTIME-OPERATIONS |
| Source | lambda_runtime_ops.go:10 |
| Mentioned In | n/a |

### policy_checks.rs:21

Inferred code symbol owned by FU-POLICY-CHECKS from policy_checks.rs:21.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-POLICY-CHECKS |
| Source | policy_checks.rs:21 |
| Mentioned In | n/a |

### policy_checks.rs:7

Inferred code symbol owned by FU-POLICY-CHECKS from policy_checks.rs:7.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-POLICY-CHECKS |
| Source | policy_checks.rs:7 |
| Mentioned In | n/a |

### pr_context_assembly.ts:13

Inferred code symbol owned by FU-PR-CONTEXT-ASSEMBLY from pr_context_assembly.ts:13.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-PR-CONTEXT-ASSEMBLY |
| Source | pr_context_assembly.ts:13 |
| Mentioned In | n/a |

### pr_context_assembly.ts:20

Inferred code symbol owned by FU-PR-CONTEXT-ASSEMBLY from pr_context_assembly.ts:20.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-PR-CONTEXT-ASSEMBLY |
| Source | pr_context_assembly.ts:20 |
| Mentioned In | n/a |

### pr_context_assembly.ts:6

Inferred code symbol owned by FU-PR-CONTEXT-ASSEMBLY from pr_context_assembly.ts:6.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-PR-CONTEXT-ASSEMBLY |
| Source | pr_context_assembly.ts:6 |
| Mentioned In | n/a |

### pr_context_assembly.ts:9

Inferred code symbol owned by FU-PR-CONTEXT-ASSEMBLY from pr_context_assembly.ts:9.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-PR-CONTEXT-ASSEMBLY |
| Source | pr_context_assembly.ts:9 |
| Mentioned In | n/a |

### review_orchestrator.rs:11

Inferred code symbol owned by FU-REVIEW-ORCHESTRATION from review_orchestrator.rs:11.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-REVIEW-ORCHESTRATION |
| Source | review_orchestrator.rs:11 |
| Mentioned In | n/a |

### review_orchestrator.rs:22

Inferred code symbol owned by FU-REVIEW-ORCHESTRATION from review_orchestrator.rs:22.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-REVIEW-ORCHESTRATION |
| Source | review_orchestrator.rs:22 |
| Mentioned In | n/a |

### review_orchestrator.rs:6

Inferred code symbol owned by FU-REVIEW-ORCHESTRATION from review_orchestrator.rs:6.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-REVIEW-ORCHESTRATION |
| Source | review_orchestrator.rs:6 |
| Mentioned In | n/a |

### review_publication.ts:13

Inferred code symbol owned by FU-REVIEW-PUBLICATION from review_publication.ts:13.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-REVIEW-PUBLICATION |
| Source | review_publication.ts:13 |
| Mentioned In | n/a |

### review_publication.ts:27

Inferred code symbol owned by FU-REVIEW-PUBLICATION from review_publication.ts:27.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-REVIEW-PUBLICATION |
| Source | review_publication.ts:27 |
| Mentioned In | n/a |

### review_publication.ts:33

Inferred code symbol owned by FU-REVIEW-PUBLICATION from review_publication.ts:33.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-REVIEW-PUBLICATION |
| Source | review_publication.ts:33 |
| Mentioned In | n/a |

### review_publication.ts:6

Inferred code symbol owned by FU-REVIEW-PUBLICATION from review_publication.ts:6.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-REVIEW-PUBLICATION |
| Source | review_publication.ts:6 |
| Mentioned In | n/a |

### review_publication.ts:9

Inferred code symbol owned by FU-REVIEW-PUBLICATION from review_publication.ts:9.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-REVIEW-PUBLICATION |
| Source | review_publication.ts:9 |
| Mentioned In | n/a |

### secrets_configuration.ts:10

Inferred code symbol owned by FU-SECRETS-CONFIGURATION from secrets_configuration.ts:10.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-SECRETS-CONFIGURATION |
| Source | secrets_configuration.ts:10 |
| Mentioned In | n/a |

### secrets_configuration.ts:7

Inferred code symbol owned by FU-SECRETS-CONFIGURATION from secrets_configuration.ts:7.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-SECRETS-CONFIGURATION |
| Source | secrets_configuration.ts:7 |
| Mentioned In | n/a |

#### webhook_ingress.go:13

Inferred code symbol owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go:13.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-GITHUB-WEBHOOK-INGRESS |
| Source | webhook_ingress.go:13 |
| Mentioned In | n/a |

#### webhook_ingress.go:17

Inferred code symbol owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go:17.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-GITHUB-WEBHOOK-INGRESS |
| Source | webhook_ingress.go:17 |
| Mentioned In | n/a |

#### webhook_ingress.go:28

Inferred code symbol owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go:28.

| Key | Value |
| --- | --- |
| Kind | Code symbol |
| Owner | FU-GITHUB-WEBHOOK-INGRESS |
| Source | webhook_ingress.go:28 |
| Mentioned In | n/a |

## 14.5. Verification References

#### VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5

exercises end-to-end pull request opened flow from webhook ingest to published review

| Key | Value |
| --- | --- |
| Kind | e2e |
| Status | pass |
| Source | test-results/contract-checkrun.json (+2) |
| Name | Pr Opened Review Flow |
| Mentioned In | Interaction Flow View , Pr Opened Review Flow |

VER-INF-INTEGRATION-AUDIT_REDACTION_TEST-B108A1

checks audit evidence redaction for secrets and sensitive payload fields

| Key | Value |
| --- | --- |
| Kind | integration |
| Status | not-run |
| Source | tests/integration/audit_redaction_test.ts |
| Name | Audit Redaction Test |
| Mentioned In | Interaction Flow View , Audit Redaction Test |

VER-INF-INTEGRATION-POLICY_ONLY_FALLBACK_TEST-76E20A

verifies policy-only fallback when Bedrock inference is unavailable

| Key | Value |
| --- | --- |
| Kind | integration |
| Status | pass |
| Source | test-results/integration-policy-fallback.json (+1) |
| Name | Policy Only Fallback Test |
| Mentioned In | Interaction Flow View , Policy Only Fallback Test |

VER-INF-TEST-GITHUB_CHECK_RUN_CONTRACT_TEST-17DBA6

validates GitHub check-run contract shape and required review result fields

| Key | Value |
| --- | --- |
| Kind | test |
| Status | not-run |

| Key | Value |
| --- | --- |
| Source | tests/contract/github_check_run_contract_test.go |
| Name | Github Check Run Contract Test |
| Mentioned In | Interaction Flow View , Github Check Run Contract Test |

### VER-INF-UNIT-RATE_LIMIT_DEFERRAL_TEST-DC9EB2

verifies rate-limit deferral behavior for pull request review processing

| Key | Value |
| --- | --- |
| Kind | unit |
| Status | pass |
| Source | test-results/unit-rate-limit.xml (+1) |
| Name | Rate Limit Deferral Test |
| Mentioned In | Interaction Flow View , Rate Limit Deferral Test |

# 15. Gemara GRC Model

This chapter renders the model as OpenSSF Gemara (https://gemara.openssf.org) documents. Each artifact is built with the official `go-gemara` SDK types and validated against the published Gemara CUE schemas. Generate the full YAML set with `enggemara`; query it via the `gemara.*` MCP tools.

| Gemara Layer | Artifact | Entries |
|---|---|---|
| L1 | Vector Catalog | 4 |
| L1 | Principle Catalog | 5 |
| L1 | Guidance Catalog | 2 |
| L2 | Capability Catalog | 8 |
| L2 | Control Catalog | 2 |
| L2 | Threat Catalog | 3 |
| L3 | Risk Catalog | 3 |
| L3 | Policy | 3 |
| - | Control→Threat Mapping | 2 |
| - | Lexicon | 39 |
| L7 | Audit Log | 6 |
| L6 | Enforcement Log | 1 |

## 15.1. Controls (L2)

| Control | Objective | Group | Assessment Requirements |
|---|---|---|---|
| CTRL-BEDROCK-WEBHOOK-SIGNATURE | Enforce signature verification for all GitHub webhook events. | input-integrity | 2 |
| CTRL-BEDROCK-SECRETS-ROTATION | Rotate app and model credentials according to policy. | credential-management | 1 |

## 15.2. Threats (L2)

| Threat | Title | Group |
|---|---|---|
| TS-BEDROCK-WEBHOOK-SPOOF | Spoofed webhook bypasses ingress verification | stride-spoofing |
| TS-BEDROCK-PROMPT-INJECTION | Prompt injection in diff influences review output integrity | stride-tampering |

| Threat | Title | Group |
| --- | --- | --- |
| TS-BEDROCK-SECRET-LEAK-IN-PUBLICATION | Sensitive tokens leak via review publication payload | stride-information-disclosure |

## 15.3. Risks (L3)

| Risk | Title | Severity |
| --- | --- | --- |
| RISK-BEDROCK-WEBHOOK-SPOOFING | Webhook spoofing risk | High |
| RISK-BEDROCK-PROMPT-INJECTION | Prompt injection in PR context risk | Medium |
| RISK-BEDROCK-SECRET-LEAKAGE | Review output secret leakage risk | Medium |